claude-windows / df5d5fb2-534a-447a-8181-b222f94655f3

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\df5d5fb2-534a-447a-8181-b222f94655f3\subagents\workflows\wf_f6b96576-946\agent-a41b9be5b9c279ad6.jsonl`
- SHA-256: `cff5f8a9c554da79eaf6924f867a94cca2f5454eb82eebdbed87e8c32b836b07`
- Source modified: `2026-07-06T18:05:02+00:00`
- Imported at: `2026-07-08T15:59:35+00:00`
- project: `wf_f6b96576-946`
- session_id: `df5d5fb2-534a-447a-8181-b222f94655f3`

Transcript

1. user (2026-07-06T18:00:43.633Z)

```
You are a CONVENTION & TEST-QUALITY reviewer.
REPO ROOT: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
FULL DIFF UNDER REVIEW (read first): C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-
Projects-badminton-highlight-
indexer/df5d5fb2-534a-447a-8181-b222f94655f3/scratchpad/wf294_full.diff

BACKGROUND: wf294 = the Cloud Run rally-worker recorded status="success" / 0 segments even when
the WASB detector TIMED OUT and produced no trajectory ? a detector failure indistinguishable
from
a legit 0-rally video. An earlier change (already reviewed CLEAN) fixed the two core detector
paths
(healthcheck-not-ready, no-trajectory-abort) to return backend.results.Failure, and made
worker.cmd_infer's _fail() write+push a status="failed" report.json + done-marker.

THIS REVIEW covers the FOLLOW-UP completeness work that closed the same silent-failure CLASS on
adjacent paths (a prior review confirmed these were the last gaps). Three sub-changes:

F1 ? backend/pipeline/segmenters/trajectory_hybrid.py process_video: the pre-detection
config/env/
   input abort paths now return Failure instead of bare []: unknown sport (KeyError), missing
API
   key (skip_ai=False), unknown provider (KeyError), invalid/<=0 video duration, and
_resolve_runner
   NotImplementedError (detector_impl=native on a family with no native runner). A genuine empty
   match (trajectory produced but zero candidate windows) still returns bare [].

F2 ? backend/pipeline/segmenters/yolo_hybrid.py (the model-default segmenter): same treatment ?
   _resolve_provider now returns Failure (not None) on bad sport / missing key / unknown
provider;
   process_video returns Failure on invalid duration; return annotation widened; imports
Failure.

F3 ? the AI-handoff "all windows errored" path in BOTH trajectory_hybrid.py (inline Phase 3) and
   yolo_hybrid.py (_run_ai_handoff): the per-window handoff fn now returns (valid_segments,
errored)
   where errored=True iff the window yielded NO verdict (clip extraction failed OR provider
   metrics['error']). If candidate windows > 0 AND every handed-off window errored,
process_video
   returns a Failure(is_recoverable=True). If ANY window ran (even returning no rallies), it
stays a
   Success (bare list). skip_ai (fully-local) has no provider and never reaches this path.

CONTRACT: every caller (worker.cmd_infer:326, main.py:1557, orchestration.py:806) branches on
isinstance(result, Failure). worker._fail writes status="failed" report.json + marker.
```

```
trajectory_hybrid is NOT mypy-quarantined (must stay mypy-clean); yolo_hybrid + worker.__main__
ARE
quarantined (mypy.ini ignore_errors). CI: ruff 0.15.16, mypy 2.1.0 (mypy backend training),
run_all_tests.py, coverage floor 70%. Current state: ruff clean, mypy clean, 1740 passed / 3
skipped, coverage 85.85%. So do NOT speculate that "tests might fail" ? cite concrete code
defects.

Report ONLY real defects with exact file:line: a wrong/edge behavior (off-by-one, an all-errored
run that stays "success", or a partial/legit-empty run wrongly failed), a broken contract, a
thread-safety/ordering bug, a regression vs the prior behavior, a convention violation, or a
test
that asserts the wrong thing / has a gap. If you cannot ground a concern in the actual code, DO
NOT
report it.

YOUR LENS:
- Convention: is the Failure usage consistent with gemini/motion (message text, is_recoverable)?
  is_recoverable: config errors default False, missing-key True, detector/handoff transient True
?
  is that coherent and is is_recoverable read by any consumer (should it be)? Are the widened
return
  annotations + any # type: ignore correct given the quarantine map? Any stale comment/docstring
  now contradicted by the code?
- Tests: do the new/updated tests actually LOCK F1/F2/F3?
  * F3: is there a test that a PARTIAL error (some windows error, >=1 runs) stays a Success, AND
a
    test that ALL-errored -> Failure, for BOTH trajectory (parametrized twins) and yolo? Are the
    assertions specific (isinstance Failure + is_recoverable + a real segment on the success
side)?
  * F1/F2: is the missing-key path asserted as Failure in both tracknet and yolo tests? Are the
    other converted paths (unknown sport/provider, invalid duration, native-not-supported)
covered
    at all, or silently uncovered?
  * Any test still asserting == [] on a path that now returns Failure (would be a latent wrong
    assertion that only passes because a mock doesn't trigger it)?
- Is the yolo partial test's mocking (patching _collect_candidate_windows/_persist_candidates)
  sound, or does it bypass so much that it doesn't actually exercise _run_ai_handoff's new
logic?
Report concrete convention violations or test gaps with file:line.
```

2. assistant (2026-07-06T18:00:48.955Z)

```
I'll start by reading the full diff under review, then examine the actual source files and
tests.
```

3. user (2026-07-06T18:00:50.879Z)

```
1       diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
2       index 503f6d8..7987278 100644
3       --- a/backend/pipeline/segmenters/trajectory_hybrid.py
4       +++ b/backend/pipeline/segmenters/trajectory_hybrid.py
5       @@ -22,7 +22,7 @@ import concurrent.futures
6        import logging
7        import os
8        import time
9       -from typing import Any, Dict, List, Optional, Tuple
10      +from typing import Any, Dict, List, Optional, Tuple, Union
11
12       import cv2
13
14      @@ -30,6 +30,9 @@ from backend.config.models import IndexingConfig
15       from backend.pipeline.detectors.base import DetectorRunner
16       from backend.pipeline.segmenters.base import BasePlaySegmenter
17       from backend.providers import provider_registry
```

```
18      +
19      +# Standardized Success|Failure contract: a detector Failure is NOT a 0-rally result.
20      +from backend.results import Failure
21       from backend.sports import sport_registry
22       from backend.utils.paths import output_base
23       from backend.utils.video import extract_video_chunk, get_video_duration
24      @@ -174,10 +177,14 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
25               video_path: str,
26               player_pool: Optional[List[str]] = None,
27               max_frames: Optional[int] = None,
28      -    ) -> List[Dict[str, Any]]:
29      -        # override-ignore: the ABC declares the Result contract (Success|Failure)
30      -        # but every live segmenter still returns a bare list ? the documented
31      -        # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
32      +    ) -> "Union[List[Dict[str, Any]], Failure]":
33      +        # Result contract (the ABC declares Success|Failure). Anything that means the
run could
34      +        # NOT actually complete ? an unrunnable config (unknown sport/provider, missing
API key),
35      +        # an unreadable video, an unavailable detector (native-not-supported), the env
healthcheck
36      +        # not ready, or run_predict timing out / aborting with no trajectory ? returns
a ``Failure``
37      +        # so it is NOT confused with a genuine 0-rally video. A LEGITIMATE empty match
(a trajectory
38      +        # was produced but no candidate/AI-confirmed rallies) still returns a bare
``[]``. Every
39      +        # caller (worker.cmd_infer, main.py, orchestration) branches on
isinstance(result, Failure).
40               label = self.display_label
41               # --- Sport profile + AI provider wiring (identical across segmenters) ---
42               sport_name = self.config.sport
43      @@ -185,7 +192,8 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
44                   sport_profile = sport_registry.get(sport_name)
45               except KeyError as e:
46                   logger.error(str(e))
47      -            return []
48      +            # Unrunnable config, not a 0-rally match (mirrors the gemini segmenter).
49      +            return Failure(str(e))
50
51               idx_cfg = self.config.indexing
52               skip_ai = bool(idx_cfg.skip_ai_handoff)
53      @@ -205,24 +213,31 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
54                   if not api_key:
55                       from backend.pipeline.segmenters._keyhint import api_key_hint
56
57      -                logger.error(
58      +                msg = (
59                           f"{api_key_env_var} environment variable is not set! "
60                           f"To run the {provider_name} handoff, set your API key. "
61                           + api_key_hint(api_key_env_var)
62                       )
63      -                return []
64      +                logger.error(msg)
65      +                # A misconfig (unrunnable), NOT a 0-rally match ? surface as Failure so
the run is
66      +                # marked failed instead of a silent empty "success" (mirrors gemini;
the worker's
67      +                # own 0-segment diagnostic names this missing-key case as a top cause).
68      +                return Failure(msg, is_recoverable=True)
69                   try:
70                       provider = provider_registry.get(
71                           provider_name, api_key=api_key, model_name=model_name
72                       )
73                   except KeyError as e:
```

```
                        logger.error(str(e))
-                       return []
+                       return Failure(str(e))

                video_duration = get_video_duration(video_path)
                if video_duration <= 0:
                    logger.error("Invalid video duration.")
-                   return []
+                   # An unreadable/broken video, not a 0-rally match.
+                   return Failure(
+                       message=f"{label}: invalid/unreadable video duration
({video_duration}).",
+                   )
                fps, frame_width = self._probe_fps_width(video_path)

                # --- Runner config + windowing thresholds ---
    @@ -233,9 +248,10 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
                    runner, runner_params = self._resolve_runner(idx_cfg)
                except NotImplementedError as e:
                    # e.g. detector_impl="native" for a family without a native runner
(tracknet, or
-                   # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of
crashing.
+                   # fusion with a tracknet substrate). An unavailable detector is a failure,
not a
+                   # 0-rally match ? refuse cleanly as a Failure instead of crashing or a
false success.
                    logger.error("%s: %s", label, e)
-                   return []
+                   return Failure(message=f"{label}: {e}")

                velocity_thresh = getattr(idx_cfg, self.family).velocity_threshold
                merge_gap = idx_cfg.dead_time_merge_gap
    @@ -267,7 +283,12 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
                ok, msg = runner.healthcheck()
                if not ok:
                    logger.error("%s detector environment not ready: %s", label, msg)
-                   return []
+                   # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
+                   # failed/retryable instead of writing an empty-but-"successful" result.
+                   return Failure(
+                       message=f"{label} detector environment not ready: {msg}",
+                       is_recoverable=True,
+                   )
                logger.info("%s detector env OK: %s", label, msg)

                # --- Phase 2: trajectory -> candidate rally windows ---
    @@ -278,7 +299,17 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
                csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
                if not csv_path:
                    logger.error("%s produced no trajectory; aborting.", label)
-                   return []
+                   # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
+                   # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
+                   # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
+                   # rather than a silent 0-segment "success" (the wf294 incident).
+                   return Failure(
+                       message=(
+                           f"{label} detector produced no trajectory (timed out or aborted) ?
"
+                           "detector failure, not a 0-rally video."
```

```
    +                ),
    +                is_recoverable=True,
    +            )

             points = runner.parse_trajectory_csv(csv_path)
             logger.info(
@@ -387,7 +418,11 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
                     len(handoff_indices),
                 )

-            def process_candidate_window(wi: int, window: Dict[str, Any]) ->
List[dict]:
+            def process_candidate_window(
+                wi: int, window: Dict[str, Any]
+            ) -> Tuple[List[dict], bool]:
+                """Returns (valid_segments, errored) ? errored=True when this window
yielded NO
+                verdict (clip extraction failed, or the provider returned
metrics['error'])."""
                     w_start = max(0, window["start"] - handoff_padding)
                     w_end = min(video_duration, window["end"] + handoff_padding)
                     w_dur = w_end - w_start
@@ -395,7 +430,7 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
                     if not extract_video_chunk(
                         video_path, w_start, w_dur, w_path, reencode=True
                     ):
-                        return []
+                        return [], True  # extraction failed ? no verdict for this window
                     prompt = sport_profile.get_prompt(chunk_duration=w_dur)
                     segments, metrics = provider.analyze_video(
                         w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
@@ -403,7 +438,8 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
                     # B2: a provider failure returns [] with metrics["error"] set ? that is
                     # "we don't know", NOT "no rallies in this window". Surface it loudly
so a
                     # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
-                    if metrics.get("error"):
+                    errored = bool(metrics.get("error"))
+                    if errored:
                         logger.warning(
                             "[Handoff %d] provider error ? window %d skipped: %s",
                             wi + 1,
@@ -427,7 +463,7 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
                         os.remove(w_path)
                     except OSError:
                         pass
-                return valid
+                return valid, errored

             with concurrent.futures.ThreadPoolExecutor(
                 max_workers=ai_max_workers
@@ -438,8 +474,10 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
                         )
                         for wi in handoff_indices
                     ]
-                    for future in futures:
-                        ai_segments.extend(future.result())
+                    window_results = [future.result() for future in futures]
+            for valid, _errored in window_results:
+                ai_segments.extend(valid)
+            handoff_failed = sum(1 for _valid, errored in window_results if errored)
             try:
                 os.rmdir(handoff_dir)
             except OSError:
```

```
    @@ -448,6 +486,18 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
                        "Phase 3 completed. AI confirmed %d semantic play segments.",
                        len(ai_segments),
                    )
    +            # wf294 (all-errored handoff): if EVERY handed-off window failed to yield a
verdict
    +            # (provider error / clip extraction failed), the run has zero information ?
a FAILURE,
    +            # not a 0-rally match. A single window that RAN and returned no rallies is
a legit empty
    +            # and keeps this from firing. (skip_ai has no provider, so it never reaches
here.)
    +            if handoff_indices and handoff_failed == len(handoff_indices):
    +                msg = (
    +                    f"{label} AI handoff produced no verdict: all {handoff_failed}
candidate "
    +                    "window(s) errored (provider error / clip extraction failed) ?
incomplete "
    +                    "run, not a 0-rally match."
    +                )
    +                logger.error(msg)
    +                return Failure(message=msg, is_recoverable=True)

                # --- Phase 4: DB population + candidate linking ---
                segments_data = []
    diff --git a/backend/pipeline/segmenters/yolo_hybrid.py
b/backend/pipeline/segmenters/yolo_hybrid.py
    index d9334f1..2e2e1fe 100644
    --- a/backend/pipeline/segmenters/yolo_hybrid.py
    +++ b/backend/pipeline/segmenters/yolo_hybrid.py
    @@ -22,11 +22,14 @@ import concurrent.futures
     import logging
     import os
     import time
    -from typing import Any, Dict, List, Tuple
    +from typing import Any, Dict, List, Tuple, Union

     from backend.pipeline.detectors.person_detector import PersonDetector
     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
     from backend.providers import provider_registry
    +
    +# Standardized Success|Failure contract: an unrunnable config is NOT a 0-rally result.
    +from backend.results import Failure
     from backend.sports import sport_registry
     from backend.utils.paths import output_base
     from backend.utils.video import extract_video_chunk, get_video_duration
    @@ -63,9 +66,10 @@ class HybridYoloSegmenter(BasePlaySegmenter):
         def _resolve_provider(self, video_path: str):
             """Phase 0: resolve sport profile + AI provider (with API key).

    -        Returns the configured provider instance, or ``None`` to signal that
    -        ``process_video`` should early-return ``[]`` (bad sport, missing API key,
    -        or unknown provider). Behaviour is identical to the inlined version.
    +        Returns the configured provider instance, or a ``Failure`` (bad sport, missing
API key,
    +        or unknown provider) that ``process_video`` propagates ? so an unrunnable
config is marked
    +        FAILED with a reason instead of a silent 0-rally "success" (the wf294 loud-
failures bar;
    +        mirrors the gemini/trajectory segmenters). A genuine "ran, found nothing" is
unaffected.
             """
             # Get sport profile config
             sport_name = self.config.sport
    @@ -73,7 +77,7 @@ class HybridYoloSegmenter(BasePlaySegmenter):
```

```
                sport_registry.get(sport_name)
            except KeyError as e:
                logger.error(str(e))
-               return None
+               return Failure(str(e))

            # Get AI provider and model config
            provider_name = self.config.ai_provider
@@ -85,12 +89,13 @@ class HybridYoloSegmenter(BasePlaySegmenter):
            if not api_key:
                from backend.pipeline.segmenters._keyhint import api_key_hint

-               logger.error(
+               msg = (
                    f"{api_key_env_var} environment variable is not set! "
                    f"To run the {provider_name} segmenter, set your API key. "
                    + api_key_hint(api_key_env_var)
                )
-               return None
+               logger.error(msg)
+               return Failure(msg, is_recoverable=True)

            try:
                provider = provider_registry.get(
@@ -98,7 +103,7 @@ class HybridYoloSegmenter(BasePlaySegmenter):
                )
            except KeyError as e:
                logger.error(str(e))
-               return None
+               return Failure(str(e))

            logger.info(
                f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}"
@@ -320,9 +325,13 @@ class HybridYoloSegmenter(BasePlaySegmenter):
            chunk_duration: float,
            handoff_padding: float,
            ai_max_workers: int,
-       ) -> List[dict]:
+       ) -> "Union[List[dict], Failure]":
            """Phase 3: AI Provider Handoff ? re-encode each padded candidate window and
ask
-       the provider for semantic boundaries, in parallel. Verbatim move."""
+       the provider for semantic boundaries, in parallel.
+
+       Returns the confirmed segments, or a ``Failure`` when EVERY candidate window
failed to
+       yield a verdict (provider error / clip extraction failed) ? an incomplete run
has zero
+       information and must NOT be recorded as a 0-rally "success" (the wf294
class)."""
            gemini_segments = []
            gemini_dir = os.path.join(
                output_base(self.config), f"chunks_{provider_name}_handoff"
@@ -333,7 +342,11 @@ class HybridYoloSegmenter(BasePlaySegmenter):
                f"Starting Phase 3: AI Provider ({provider_name}) Validation on
{len(all_candidate_windows)} candidate windows..."
            )

-       def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
+       def process_candidate_window(
+           wi: int, window: Dict[str, Any]
+       ) -> Tuple[List[dict], bool]:
+           """Returns (valid_segments, errored) ? errored=True when this window
yielded NO verdict
```

```
    +            (clip extraction failed, or the provider returned metrics['error']).""" 
                 w_start = max(0, window["start"] - handoff_padding)
                 w_end = min(video_duration, window["end"] + handoff_padding)
                 w_dur = w_end - w_start
    @@ -344,7 +357,7 @@ class HybridYoloSegmenter(BasePlaySegmenter):
                     video_path, w_start, w_dur, w_path, reencode=True
                 )
                 if not success:
    -                return []
    +                return [], True  # extraction failed ? no verdict for this window

                 prompt = sport_profile.get_prompt(chunk_duration=w_dur)
                 segments, metrics = provider.analyze_video(
    @@ -353,7 +366,8 @@ class HybridYoloSegmenter(BasePlaySegmenter):
                 # B2: a provider failure returns [] with metrics["error"] set ? that is
                 # "we don't know", NOT "no rallies in this window". Surface it loudly so a
                 # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
    -            if metrics.get("error"):
    +            errored = bool(metrics.get("error"))
    +            if errored:
                     logger.warning(
                         "[Handoff %d] provider error ? window %d skipped: %s",
                         wi + 1,
    @@ -380,7 +394,7 @@ class HybridYoloSegmenter(BasePlaySegmenter):
                 except OSError:
                     pass

    -            return valid_segments
    +            return valid_segments, errored

             with concurrent.futures.ThreadPoolExecutor(
                 max_workers=ai_max_workers
    @@ -390,8 +404,10 @@ class HybridYoloSegmenter(BasePlaySegmenter):
                     for wi, w in enumerate(all_candidate_windows)
                 ]
                 # Iterate in submission order to preserve temporal ordering
    -            for future in futures:
    -                gemini_segments.extend(future.result())
    +            window_results = [future.result() for future in futures]
    +        for valid, _errored in window_results:
    +            gemini_segments.extend(valid)
    +        handoff_failed = sum(1 for _valid, errored in window_results if errored)

             try:
                 os.rmdir(gemini_dir)
    @@ -401,6 +417,17 @@ class HybridYoloSegmenter(BasePlaySegmenter):
             logger.info(
                 f"Phase 3 completed. AI confirmed {len(gemini_segments)} semantic play
segments."
             )
    +        # wf294 (all-errored handoff): if EVERY candidate window failed to yield a
verdict
    +        # (provider error / clip extraction failed), the run has zero information ? a
FAILURE, not
    +        # a 0-rally match. A window that RAN and returned no rallies keeps this from
firing.
    +        if all_candidate_windows and handoff_failed == len(all_candidate_windows):
    +            msg = (
    +                f"yolo_hybrid AI handoff produced no verdict: all {handoff_failed}
candidate "
    +                "window(s) errored (provider error / clip extraction failed) ?
incomplete run, "
    +                "not a 0-rally match."
    +            )
```

```
359     +            logger.error(msg)
360     +            return Failure(message=msg, is_recoverable=True)
361              return gemini_segments
362
363          def _populate_db(
364     @@ -456,19 +483,23 @@ class HybridYoloSegmenter(BasePlaySegmenter):
365
366              return segments_data
367
368     -    def process_video(
369     +    def process_video(  # type: ignore[override]
370              self,
371              video_id: str,
372              video_path: str,
373              player_pool: List[str] = None,
374              max_frames: int = None,
375     -    ) -> List[Dict[str, Any]]:
376     +    ) -> "Union[List[Dict[str, Any]], Failure]":
377     +        # Result contract: an unrunnable config (bad sport/provider, missing key) or an
unreadable
378     +        # video returns a Failure so the run is marked failed ? NOT a silent 0-rally
"success"
379     +        # (the wf294 class). A genuine empty match still returns a bare []. Every
caller branches
380     +        # on isinstance(result, Failure).
381              self.person.lazy_load_model()
382
383     -        # Phase 0: resolve sport profile + AI provider (early-returns [] on any
failure).
384     +        # Phase 0: resolve sport profile + AI provider (returns a Failure on any config
failure).
385              provider = self._resolve_provider(video_path)
386     -        if provider is None:
387     -            return []
388     +        if isinstance(provider, Failure):
389     +            return provider
390
391              # Re-read the config values needed downstream (pure deterministic reads).
392              idx_cfg = self.config.indexing
393     @@ -479,7 +510,10 @@ class HybridYoloSegmenter(BasePlaySegmenter):
394              video_duration = get_video_duration(video_path)
395              if video_duration <= 0:
396                  logger.error("Invalid video duration.")
397     -            return []
398     +            # An unreadable/broken video, not a 0-rally match.
399     +            return Failure(
400     +                message=f"yolo_hybrid: invalid/unreadable video duration
({video_duration})."
401     +            )
402
403              chunk_duration = idx_cfg.chunk_duration
404              chunk_overlap = idx_cfg.chunk_overlap
405     @@ -537,8 +571,8 @@ class HybridYoloSegmenter(BasePlaySegmenter):
406              except Exception:
407                  pass
408
409     -        # Phase 3: AI Provider Handoff
410     -        gemini_segments = self._run_ai_handoff(
411     +        # Phase 3: AI Provider Handoff (returns a Failure when every window errored ?
wf294).
412     +        handoff = self._run_ai_handoff(
413                  video_id,
414                  video_path,
415                  video_duration,
416     @@ -551,9 +585,11 @@ class HybridYoloSegmenter(BasePlaySegmenter):
```

```
                handoff_padding,
                ai_max_workers,
            )
    +       if isinstance(handoff, Failure):
    +           return handoff

            # Phase 4: DB Population
    -       return self._populate_db(video_id, gemini_segments, model_name)
    +       return self._populate_db(video_id, handoff, model_name)


     # Register the segmenter
    diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
    index 461d7ca..88c50f6 100644
    --- a/backend/worker/__main__.py
    +++ b/backend/worker/__main__.py
    @@ -114,6 +114,42 @@ def _write_report_json(
             )


    +def _serialize_and_push_outputs(
    +    scratch: str,
    +    out_uri: str,
    +    video_id: str,
    +    segments: List[dict],
    +    status: str,
    +    storage_cfg: dict,
    +    video_uri: str = "",
    +) -> List[str]:
    +    """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
    +    ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
    +
    +    Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
    +    ``status="failed"`` report.json in the bucket ? the control plane's source of truth
    +    (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
    +    report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
    +    EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``."""
    +    out_local = os.path.join(scratch, "outputs", video_id)
    +    os.makedirs(out_local, exist_ok=True)
    +    _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
    +    _write_report_json(
    +        video_id,
    +        segments,
    +        status,
    +        os.path.join(out_local, "report.json"),
    +        video_uri=video_uri,
    +    )
    +    out_prefix = out_uri.rstrip("/")
    +    pushed: List[str] = []
    +    for fn in sorted(os.listdir(out_local)):
    +        uri = f"{out_prefix}/outputs/{video_id}/{fn}"
    +        storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
    +        pushed.append(uri)
    +    return pushed
    +
    +
     def _run_startup_checks(config: Any) -> bool:
         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
```

```
476     @@ -237,9 +273,26 @@ def cmd_infer(args: argparse.Namespace) -> int:
477          )
478
479          def _fail(rc: int) -> int:
480      -        # M6: on a worker-side FAILURE, write the done-marker (the REAL rc, 0 segments)
    so a remote
481      -        # dispatcher's bucket-poll terminates FAST + accurately instead of hanging
    until its poll
482      -        # budget expires. DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
    local run).
483      +        # A worker-side FAILURE (validation, unknown segmenter, or a detector
    timeout/abort the
484      +        # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
    Write + push
485      +        # a status="failed" report.json/intervals.csv (0 segments) so the control
    plane's bucket
486      +        # truth (submit-cache probe, restart-recovery) and the alpha-user result
    surface see an
487      +        # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
    must never
488      +        # mask the original failure. THEN the M6 done-marker (real rc + status) so a
    remote
489      +        # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
    expires.
490      +        try:
491      +            _serialize_and_push_outputs(
492      +                scratch,
493      +                args.out_uri,
494      +                video_id,
495      +                [],
496      +                "failed",
497      +                storage_cfg,
498      +                str(getattr(args, "video_uri", "") or ""),
499      +            )
500      +        except Exception as e:  # never mask the real failure on an artifact-push
    hiccup
501      +            logger.warning("[worker] could not push failure artifacts: %s", e)
502      +        # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
503               if getattr(args, "done_uri", None):
504                   _write_done_marker(
505                       args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
506      @@ -313,24 +366,16 @@ def cmd_infer(args: argparse.Namespace) -> int:
507               )
508
509          # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
510      -    out_local = os.path.join(scratch, "outputs", video_id)
511      -    os.makedirs(out_local, exist_ok=True)
512      -    _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
513      -    _write_report_json(
514      +    pushed = _serialize_and_push_outputs(
515      +        scratch,
516      +        args.out_uri,
517               video_id,
518               segments,
519               status,
520      -        os.path.join(out_local, "report.json"),
521      -        video_uri=str(getattr(args, "video_uri", "") or ""),
522      +        storage_cfg,
523      +        str(getattr(args, "video_uri", "") or ""),
524          )
525
526      -    out_prefix = args.out_uri.rstrip("/")
527      -    pushed = []
528      -    for fn in sorted(os.listdir(out_local)):
529      -        uri = f"{out_prefix}/outputs/{video_id}/{fn}"
```

```
-        storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
-        pushed.append(uri)
-
         print(
             f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
         )
diff --git a/tests/test_tracknet_hybrid.py b/tests/test_tracknet_hybrid.py
index 64909e7..c157ed5 100644
--- a/tests/test_tracknet_hybrid.py
+++ b/tests/test_tracknet_hybrid.py
@@ -5,6 +5,7 @@ healthcheck gating, candidate persistence, AI handoff, and DB
population/linking
 from unittest.mock import MagicMock, patch

 from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
+from backend.results import Failure


 def _window(start=1.0, end=4.0):
@@ -91,7 +92,7 @@ def test_pipeline_happy_path(


 @_patched
-def test_pipeline_no_api_key_returns_empty(
+def test_pipeline_no_api_key_is_failure(
     mock_runner_cls,
     mock_probe,
     mock_dur,
@@ -99,12 +100,16 @@ def test_pipeline_no_api_key_returns_empty(
     mock_provider_registry,
     mock_env,
 ):
+    """A missing API key (with skip_ai_handoff=False) is an unrunnable misconfig ? a
Failure, NOT a
+    0-rally []. This is a wf294-class silent-failure the worker must mark failed, not
"success"."""
     mock_env.return_value = None
     mock_dur.return_value = 30.0
     mock_runner_cls.return_value = _make_runner()

     seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
-    assert seg.process_video("v1", "clip.mp4") == []
+    res = seg.process_video("v1", "clip.mp4")
+    assert isinstance(res, Failure) and res.is_recoverable
+    assert "API_KEY" in res.message


 @_patched
@@ -124,7 +129,10 @@ def test_pipeline_healthcheck_failure_aborts(

     seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
     res = seg.process_video("v1", "clip.mp4")
-    assert res == []
+    # A detector-env failure is a Failure, NOT a 0-rally [] ? so the caller marks the
run failed
+    # instead of writing an empty "success".
+    assert isinstance(res, Failure)
+    assert "environment not ready" in res.message
     # Should bail before ever running inference.
     mock_runner_cls.return_value.run_predict.assert_not_called()

@@ -144,7 +152,36 @@ def test_pipeline_no_trajectory_aborts(
     mock_provider_registry.get.return_value = MagicMock()

```

```
590          seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
591      -    assert seg.process_video("v1", "clip.mp4") == []
592      +    res = seg.process_video("v1", "clip.mp4")
593      +    # A detector timeout/abort (run_predict yielded no trajectory) is a Failure, NOT a
0-rally [] ?
594      +    # this is exactly the wf294 silent-failure that the caller must now mark
failed/retryable.
595      +    assert isinstance(res, Failure)
596      +    assert "no trajectory" in res.message and res.is_recoverable
597      +
598      +
599      +@_patched
600      +def test_pipeline_zero_windows_is_legit_empty_not_failure(
601      +    mock_runner_cls,
602      +    mock_probe,
603      +    mock_dur,
604      +    mock_extract,
605      +    mock_provider_registry,
606      +    mock_env,
607      +):
608      +    """The other side of the wf294 distinction: the detector RAN fine (a trajectory was
produced +
609      +    parsed) but yielded no candidate rally windows. That is a legitimately empty match
? a bare []
610      +    (0-rally "success"), NOT a Failure."""
611      +    mock_env.return_value = "dummy_key"
612      +    mock_dur.return_value = 30.0
613      +    mock_extract.return_value = True
614      +    mock_runner_cls.return_value = _make_runner(windows=[])  # trajectory ok, zero
windows
615      +    mock_provider_registry.get.return_value = MagicMock()
616      +
617      +    seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
618      +    res = seg.process_video("v1", "clip.mp4")
619      +    assert res == [] and not isinstance(res, Failure)
620      +    # The detector DID run (a genuine empty, not an abort).
621      +    mock_runner_cls.return_value.run_predict.assert_called_once()
622
623
624       @_patched
625      diff --git a/tests/test_trajectory_hybrid_equivalence.py
b/tests/test_trajectory_hybrid_equivalence.py
626      index 0080f4d..0270e41 100644
627      --- a/tests/test_trajectory_hybrid_equivalence.py
628      +++ b/tests/test_trajectory_hybrid_equivalence.py
629      @@ -14,6 +14,7 @@ import pytest
630       from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
631       from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
632       from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
633      +from backend.results import Failure
634
635       TWINS = [
636           (TrackNetHybridSegmenter, "tracknet", "tracknet_hybrid"),
637      @@ -111,8 +112,10 @@ def test_provider_offsets_applied_identically(cls, family,
detector):
638
639       @pytest.mark.parametrize("cls,family,detector", TWINS)
640       def test_provider_failure_is_surfaced_not_silent(cls, family, detector, caplog):
641      -    """B2/P17: a provider failure ([], metrics['error']) must be logged loudly, not
silently
642      -    treated as 'no rallies in this window'. The run still returns [] (no analysis
done)."""
643      +    """B2/P17 + F3 (wf294): a provider failure ([], metrics['error']) must be logged
loudly, not
644      +    silently treated as 'no rallies'. When it errors on the ONLY candidate window the
```

```
whole run has
645     +    zero verdict, so process_video now returns a Failure (was a bare [] that looked
like a 0-rally
646     +    match) ? the worker marks it failed/retryable instead of a silent empty
'success'."""
647          import logging
648
649          provider = MagicMock()
650     @@ -139,13 +142,45 @@ def test_provider_failure_is_surfaced_not_silent(cls, family,
detector, caplog):
651                   logging.WARNING, logger="backend.pipeline.segmenters.trajectory_hybrid"
652              ):
653                  res = seg.process_video("v1", "clip.mp4")
654     -    assert res == []  # no analysis for the failed window
655     +    assert isinstance(res, Failure) and res.is_recoverable  # all (1) windows errored
-> no verdict
656          assert any(
657              "provider error" in r.message and "generate failed" in r.message
658              for r in caplog.records
659          ), [r.message for r in caplog.records]
660
661
662     +@pytest.mark.parametrize("cls,family,detector", TWINS)
663     +def test_partial_handoff_error_is_a_success_not_a_failure(cls, family, detector):
664     +    """The boundary of F3: when SOME candidate windows error but at least one RAN (even
if it found
665     +    no rallies), the run is NOT a failure ? only an all-errored handoff is. Here two
windows are
666     +    handed off; the first returns a rally, the second errors -> a Success with the
first rally."""
667     +    provider = MagicMock()
668     +    provider.analyze_video.side_effect = [
669     +        ([{"start_time": 0.5, "end_time": 2.0}], {}),  # window 0: a real verdict
670     +        ([], {"error": "generate failed after retries: 503"}),  # window 1: errored
671     +    ]
672     +    db = MagicMock()
673     +    db.add_candidate_segment.side_effect = [55, 56]
674     +    db.add_play_segment.return_value = 200
675     +    two_windows = [_window(1.0, 4.0), _window(10.0, 13.0)]
676     +    with patch.object(
677     +        cls, "_build_runner", return_value=(_runner(two_windows), {"weights_path":
"w"})
678     +    ), patch.object(cls, "_probe_fps_width", return_value=(30.0, 1280.0)), patch(
679     +        "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
680     +        return_value=30.0,
681     +    ), patch(
682     +        "backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
683     +        return_value=True,
684     +    ), patch(
685     +        "backend.pipeline.segmenters.trajectory_hybrid.provider_registry"
686     +    ) as preg, patch("os.environ.get", return_value="dummy_key"):
687     +        preg.get.return_value = provider
688     +        seg = cls(db, {"indexing": {}})
689     +        res = seg.process_video("v1", "clip.mp4")
690     +    assert not isinstance(res, Failure)
691     +    assert len(res) == 1 and res[0]["start_time"] == 0.5
692     +
693     +
694      def test_base_is_not_registered():
695          from backend.pipeline.segmenters.base import segmenter_registry
696
697     diff --git a/tests/test_worker.py b/tests/test_worker.py
698     index 96edbee..8fb5e16 100644
699     --- a/tests/test_worker.py
700     +++ b/tests/test_worker.py
```

```
@@ -10,6 +10,7 @@ import json
 import pytest

 from backend.config import load_config
+from backend.results import Failure
 from backend.worker import __main__ as wmain

 # ---------------------------------------------------------------- pure helpers
@@ -191,7 +192,7 @@ def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):


 class _EmptySeg:
-    """A segmenter that finds nothing ? the cold-start failure mode."""
+    """A segmenter that finds nothing ? a LEGITIMATE 0-rally match (a bare [])."""

     def __init__(self, db, config):
         pass
@@ -200,6 +201,20 @@ class _EmptySeg:
         return []


+class _FailSeg:
+    """A segmenter whose DETECTOR failed/timed out ? returns a Failure (e.g. WASB
produced no
+    trajectory), which is NOT the same as a 0-rally match. This is the wf294 shape."""
+
+    def __init__(self, db, config):
+        pass
+
+    def process_video(self, video_id, path, max_frames=None):
+        return Failure(
+            message="WASB detector produced no trajectory (timed out or aborted)",
+            is_recoverable=True,
+        )
+
+
 def test_setup_logging_levels():
     import logging

@@ -246,12 +261,112 @@ def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path,
monkeypatch, caplog)
     assert "0 segments" in msgs and "vidZ" in msgs
     assert "detector_impl=native" in msgs  # surfaces the resolved detector
     assert "native runner UNAVAILABLE" in msgs  # points at the likely cause
-    assert (
-        json.loads((out / "outputs" / "vidZ" / "report.json").read_text())[
-            "segment_count"
+    # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match ? status stays
"success"
+    # (contrast test_cmd_infer_detector_failure_*, where a Failure surfaces as
status="failed").
+    report = json.loads((out / "outputs" / "vidZ" / "report.json").read_text())
+    assert report["segment_count"] == 0 and report["status"] == "success"
+
+
+def test_cmd_infer_detector_failure_marks_report_and_marker_failed(
+    tmp_path, monkeypatch
+):
+    """wf294 fix: a segmenter that returns a Failure (detector timeout/abort) must NOT
be written
+    as a 0-segment "success". cmd_infer returns rc 3 and writes report.json + the done-
marker with
+    status="failed" so the control plane / alpha-user surface can show an error /
retry."""
+    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidX")
```

```
759     +    monkeypatch.setattr(
760     +        wmain.validator_registry,
761     +        "run_all_with_context",
762     +        lambda path, cfg: ([_OKResult()], {}),
763     +    )
764     +    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FailSeg)
765     +    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
766     +    video = tmp_path / "in.mp4"
767     +    video.write_bytes(b"FAKE")
768     +    out = tmp_path / "bucket"
769     +    done = tmp_path / "bucket" / "_dispatch" / "tok.done"
770     +
771     +    rc = wmain.main(
772     +        [
773     +            "infer",
774     +            "--video-uri",
775     +            str(video),
776     +            "--out-uri",
777     +            str(out),
778     +            "--scratch",
779     +            str(tmp_path / "s"),
780     +            "--segmenter",
781     +            "wasb_hybrid",
782     +            "--done-uri",
783     +            str(done),
784     +        ]
785     +    )
786     +    # A detector failure is rc 3 (distinct from validation's rc 2 and a clean rc 0).
787     +    assert rc == 3
788     +    # report.json is written as an explicit FAILURE, not a false empty "success".
789     +    report = json.loads((out / "outputs" / "vidX" / "report.json").read_text())
790     +    assert report["status"] == "failed" and report["segment_count"] == 0
791     +    assert report["segments"] == []
792     +    # ...and the completion marker the cloud dispatcher bucket-polls carries the same
verdict.
793     +    marker = json.loads(done.read_text())
794     +    assert marker["returncode"] == 3 and marker["status"] == "failed"
795     +    assert marker["segment_count"] == 0 and marker["video_id"] == "vidX"
796     +
797     +
798     +def test_cmd_infer_real_trajectory_segmenter_failure_flows_to_report(tmp_path,
monkeypatch):
799     +    """End-to-end: the ACTUAL WasbHybridSegmenter no-trajectory abort (the wf294 shape)
flows
800     +    through cmd_infer -> _fail(3) -> a status="failed" report.json + done-marker. Locks
the two
801     +    halves (segmenter Failure + worker report/marker) TOGETHER, not just via the
_FailSeg stub."""
802     +    from unittest.mock import MagicMock
803     +
804     +    from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
805     +
806     +    runner = MagicMock()
807     +    runner.healthcheck.return_value = (True, "env ok")
808     +    runner.run_predict.return_value = None  # WASB timed out / aborted: no trajectory
produced
809     +
810     +    # Force the real base process_video down to its no-trajectory abort without a
GPU/video.
811     +    monkeypatch.setattr(WasbHybridSegmenter, "_resolve_runner", lambda self, cfg:
(runner, {}))
812     +    monkeypatch.setattr(
813     +        WasbHybridSegmenter, "_probe_fps_width", lambda *a, **k: (30.0, 1280.0)
814     +    )
815     +    monkeypatch.setattr(
```

```
816     +        "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration", lambda p:
30.0
817     +    )
818     +    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidR")
819     +    monkeypatch.setattr(
820     +        wmain.validator_registry, "run_all_with_context", lambda path, cfg:
([_OKResult()], {})
821     +    )
822     +    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name:
WasbHybridSegmenter)
823     +    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
824     +
825     +    video = tmp_path / "in.mp4"
826     +    video.write_bytes(b"FAKE")
827     +    out = tmp_path / "bucket"
828     +    done = tmp_path / "bucket" / "_dispatch" / "tok.done"
829     +
830     +    rc = wmain.main(
831     +        [
832     +            "infer",
833     +            "--video-uri",
834     +            str(video),
835     +            "--out-uri",
836     +            str(out),
837     +            "--scratch",
838     +            str(tmp_path / "s"),
839     +            "--segmenter",
840     +            "wasb_hybrid",
841     +            "--done-uri",
842     +            str(done),
843     +            "--set",
844     +            "indexing.skip_ai_handoff=true",  # no API key needed; isolate the detector
abort
845             ]
846     -       == 0
847         )
848     +    assert rc == 3
849     +    report = json.loads((out / "outputs" / "vidR" / "report.json").read_text())
850     +    assert report["status"] == "failed" and report["segment_count"] == 0
851     +    marker = json.loads(done.read_text())
852     +    assert marker["returncode"] == 3 and marker["status"] == "failed"
853
854
855      def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
856     @@ -266,20 +381,30 @@ def test_cmd_infer_validation_failure_returns_2(tmp_path,
monkeypatch):
857         video.write_bytes(b"X")
858         cfg = tmp_path / "config.json"
859         cfg.write_text("{}")
860     +    out = tmp_path / "o"
861     +    done = tmp_path / "o" / "_dispatch" / "tok.done"
862         rc = wmain.main(
863             [
864                 "infer",
865                 "--video-uri",
866                 str(video),
867                 "--out-uri",
868     -           str(tmp_path / "o"),
869     +           str(out),
870                 "--config",
871                 str(cfg),
872                 "--scratch",
873                 str(tmp_path / "s"),
874     +           "--done-uri",
875     +           str(done),
```

```
            ]
        )
        assert rc == 2
    +    # A validation failure is ALSO written as an explicit failure (not a missing
artifact / false
    +    # success): report.json status="failed" + the done-marker rc 2 / status="failed".
    +    report = json.loads((out / "outputs" / "vidF" / "report.json").read_text())
    +    assert report["status"] == "failed" and report["segment_count"] == 0
    +    marker = json.loads(done.read_text())
    +    assert marker["returncode"] == 2 and marker["status"] == "failed"


     # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push
bundle ---
    diff --git a/tests/test_yolo_hybrid.py b/tests/test_yolo_hybrid.py
    index 38e78b2..09da6c2 100644
    --- a/tests/test_yolo_hybrid.py
    +++ b/tests/test_yolo_hybrid.py
    @@ -11,6 +11,7 @@ import os
     from unittest.mock import MagicMock, patch

     from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
    +from backend.results import Failure


     def _make_cap_mock(num_frames=300):
    @@ -110,8 +111,10 @@ def test_yolo_hybrid_segmenter(
     def test_yolo_hybrid_provider_failure_is_surfaced_not_silent(
         mock_env_get, mock_provider_registry, mock_extract, mock_get_dur, mock_cap, caplog
     ):
    -    """B2/P17: a provider failure ([], metrics['error']) must be logged loudly, not
silently
    -    treated as 'no rallies'. The run still returns [] (no analysis done)."""
    +    """B2/P17 + F3 (wf294): a provider failure ([], metrics['error']) must be logged
loudly, not
    +    silently treated as 'no rallies'. When it errors on the ONLY candidate window the
whole run has
    +    zero verdict, so process_video now returns a Failure (was a bare [] that looked
like a 0-rally
    +    match) ? the worker marks it failed/retryable instead of a silent empty
'success'."""
         import logging

         mock_env_get.return_value = "dummy_api_key"
    @@ -147,13 +150,49 @@ def test_yolo_hybrid_provider_failure_is_surfaced_not_silent(
             logging.WARNING, logger="backend.pipeline.segmenters.yolo_hybrid"
         ):
             res = segmenter.process_video("v1", "dummy.mp4")
    -    assert res == []  # no analysis for the failed window
    +    assert isinstance(res, Failure) and res.is_recoverable  # all (1) windows errored
-> no verdict
         assert any(
             "provider error" in r.message and "generate failed" in r.message
             for r in caplog.records
         ), [r.message for r in caplog.records]


    +@patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
    +@patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
    +@patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
    +@patch("os.environ.get")
    +def test_yolo_hybrid_partial_handoff_error_is_a_success_not_a_failure(
    +    mock_env_get, mock_provider_registry, mock_extract, mock_get_dur
    +):
    +    """The boundary of F3: SOME windows errored but at least one RAN, so the run is a
```

```
Success (the
933     +    first window's rally), NOT a Failure. Stage 2 is patched to hand off two windows
directly."""
934     +    mock_env_get.return_value = "dummy_api_key"
935     +    mock_get_dur.return_value = 30.0
936     +    mock_extract.return_value = True
937     +
938     +    mock_provider = MagicMock()
939     +    mock_provider.analyze_video.side_effect = [
940     +        ([{"start_time": 0.5, "end_time": 2.0}], {}),  # window 0: a real verdict
941     +        ([], {"error": "generate failed after retries: 503"}),  # window 1: errored
942     +    ]
943     +    mock_provider_registry.get.return_value = mock_provider
944     +
945     +    db_mock = MagicMock()
946     +    db_mock.add_play_segment.return_value = "yolo_segment_1"
947     +    two_windows = [{"start": 1.0, "end": 4.0}, {"start": 10.0, "end": 13.0}]
948     +
949     +    segmenter = HybridYoloSegmenter(db_mock, {"indexing": {"use_gpu": False}})
950     +    segmenter.person.model = MagicMock()  # short-circuit lazy model load
951     +    with patch.object(
952     +        HybridYoloSegmenter, "_collect_candidate_windows", return_value=two_windows
953     +    ), patch.object(
954     +        HybridYoloSegmenter, "_persist_candidates", return_value=[None, None]
955     +    ):
956     +        res = segmenter.process_video("v1", "dummy.mp4")
957     +    assert not isinstance(res, Failure)
958     +    assert len(res) == 1 and res[0]["start_time"] == 0.5
959     +
960     +
961      @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
962      @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
963      @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
964     @@ -162,7 +201,8 @@ def test_yolo_hybrid_provider_failure_is_surfaced_not_silent(
965      def test_yolo_hybrid_no_api_key(
966          mock_env_get, mock_provider_registry, mock_extract, mock_get_dur, mock_cap
967      ):
968     -    """Segmenter should return empty when GEMINI_API_KEY is missing."""
969     +    """A missing API key is an unrunnable misconfig ? a Failure, NOT a 0-rally [] that
the worker
970     +    would write as a silent status="success" (the wf294 class; yolo_hybrid is the
default segmenter)."""
971          mock_env_get.return_value = None
972
973          db_mock = MagicMock()
974     @@ -172,7 +212,8 @@ def test_yolo_hybrid_no_api_key(
975          segmenter.person.model = MagicMock()  # short-circuit lazy model load
976
977          res = segmenter.process_video("v1", "dummy.mp4")
978     -    assert res == []
979     +    assert isinstance(res, Failure) and res.is_recoverable
980     +    assert "API_KEY" in res.message
981
982
983      @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
984
```

4. assistant (2026-07-06T18:00:53.810Z)

I have read the full diff. Now let me examine the actual source files and tests to verify the behavior claims, especially around the F3 all-errored logic, the yolo partial test mocking, and the worker `_fail` path.

5. user (2026-07-06T18:00:55.831Z)

```
            }

            # --- Phase 1: env healthcheck (fail fast with a clear message) ---
            ok, msg = runner.healthcheck()
            if not ok:
                logger.error("%s detector environment not ready: %s", label, msg)
                # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
                # failed/retryable instead of writing an empty-but-"successful" result.
                return Failure(
                    message=f"{label} detector environment not ready: {msg}",
                    is_recoverable=True,
                )
            logger.info("%s detector env OK: %s", label, msg)

            # --- Phase 2: trajectory -> candidate rally windows ---
            # Trajectory detectors process the whole video in one pass (they carry
            # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
            t_start = time.time()
            out_dir = os.path.join(output_base(self.config), self.family)
            csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
            if not csv_path:
                logger.error("%s produced no trajectory; aborting.", label)
                # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
                # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
                # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
                # rather than a silent 0-segment "success" (the wf294 incident).
                return Failure(
                    message=(
                        f"{label} detector produced no trajectory (timed out or aborted) ? "
                        "detector failure, not a 0-rally video."
                    ),
                    is_recoverable=True,
                )

            points = runner.parse_trajectory_csv(csv_path)
            logger.info(
                "Parsed %d trajectory points (%d visible).",
                len(points),
                sum(1 for p in points if p.visible),
            )

            all_candidate_windows = runner.trajectory_to_action_windows(
                points,
                fps=fps,
                frame_width=frame_width,
                chunk_start=0.0,
                velocity_thresh=velocity_thresh,
                merge_gap=merge_gap,
                min_window_duration=min_window_duration,
                chunk_index=0,
                max_window_duration=max_window_duration,
                min_split_gap=window_min_split_gap,
                window_hard_cap=window_hard_cap,
                window_inactivity_end=window_inactivity_end,
                inactivity_rally_thresh=inactivity_rally_thresh,
                inactivity_min_density=inactivity_min_density,
                inactivity_temporal_density=inactivity_temporal_density,
                window_blob_fallback=window_blob_fallback,
                window_blob_factor=window_blob_factor,
            )
            logger.info(
```

```
                "Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
                label,
                time.time() - t_start,
                len(all_candidate_windows),
            )

            if log_candidates:
                for cw in all_candidate_windows:
                    logger.info(
                        f"[{label} rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
                        f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
                        f"peak_v={cw['peak_velocity']:.3f} mean_v={cw['mean_velocity']:.3f}"
                    )

            # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
            # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
            # persistence and the handoff gate below.
            window_stats = [
                self._enrich_candidate_stats(
                    cw, points, fps, frame_width, video_path, idx_cfg
                )
                for cw in all_candidate_windows
            ]

            # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
            candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
            if store_candidates:
                for i, cw in enumerate(all_candidate_windows):
                    candidate_ids[i] = self.db.add_candidate_segment(
                        video_id,
                        detector=self.name,
                        start_time=cw["start"],
                        end_time=cw["end"],
                        chunk_index=cw["chunk_index"],
                        confidence=min(1.0, cw["peak_velocity"]),
                        stats=window_stats[i],
                        detection_params=detection_params,
                    )

            if not all_candidate_windows:
                return []

            # --- Phase 3: AI provider handoff over padded candidate clips ---
            # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
            # candidates above stay the full superset ? only the served play_segments are
gated.
            handoff_indices = self._select_for_handoff(
                all_candidate_windows, window_stats, idx_cfg
            )

            ai_segments: List[dict] = []
            if skip_ai:
                # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
                # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
                # falls back to the duration-based estimate (no AI exchange count).
                ai_segments = [
                    {
                        "start_time": all_candidate_windows[wi]["start"],
                        "end_time": all_candidate_windows[wi]["end"],
```

```
                        "shuttle_exchange_count": None,
                        "shot_count_confidence": "LocalNoAI",
                        "_candidate_id": candidate_ids[wi],
                    }
                    for wi in handoff_indices
                ]
                logger.info(
                    "Phase 3 SKIPPED (fully-local): emitted %d candidate windows as rallies
"
                    "(no AI handoff).",
                    len(ai_segments),
                )
            else:
                handoff_dir = os.path.join(
                    output_base(self.config), f"chunks_{provider_name}_handoff"
                )
                os.makedirs(handoff_dir, exist_ok=True)
                logger.info(
                    "Phase 3: %s validation on %d candidate windows...",
                    provider_name,
                    len(handoff_indices),
                )

                def process_candidate_window(
                    wi: int, window: Dict[str, Any]
                ) -> Tuple[List[dict], bool]:
                    """Returns (valid_segments, errored) ? errored=True when this window
yielded NO
                    verdict (clip extraction failed, or the provider returned
metrics['error'])."""
                    w_start = max(0, window["start"] - handoff_padding)
                    w_end = min(video_duration, window["end"] + handoff_padding)
                    w_dur = w_end - w_start
                    w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
                    if not extract_video_chunk(
                        video_path, w_start, w_dur, w_path, reencode=True
                    ):
                        return [], True  # extraction failed ? no verdict for this window
                    prompt = sport_profile.get_prompt(chunk_duration=w_dur)
                    segments, metrics = provider.analyze_video(
                        w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
                    )
                    # B2: a provider failure returns [] with metrics["error"] set ? that is
                    # "we don't know", NOT "no rallies in this window". Surface it loudly so
a
                    # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
                    errored = bool(metrics.get("error"))
                    if errored:
                        logger.warning(
                            "[Handoff %d] provider error ? window %d skipped: %s",
                            wi + 1,
                            wi,
                            metrics["error"],
                        )
                    valid = []
                    for seg in segments:
                        try:
                            seg["start_time"] = float(seg["start_time"]) + w_start
                            seg["end_time"] = float(seg["end_time"]) + w_start
                            seg["_candidate_id"] = candidate_ids[wi]
                            valid.append(seg)
                        except (ValueError, TypeError, KeyError) as e:
                            logger.warning(
                                "Dropping segment with unparseable timestamps: %s ? %s",
```

```
                                seg,
                                e,
                            )
                    try:
                        os.remove(w_path)
                    except OSError:
                        pass
                    return valid, errored

                with concurrent.futures.ThreadPoolExecutor(
                    max_workers=ai_max_workers
                ) as executor:
                    futures = [
                        executor.submit(
                            process_candidate_window, wi, all_candidate_windows[wi]
                        )
                        for wi in handoff_indices
                    ]
                    window_results = [future.result() for future in futures]
                for valid, _errored in window_results:
                    ai_segments.extend(valid)
                handoff_failed = sum(1 for _valid, errored in window_results if errored)
                try:
                    os.rmdir(handoff_dir)
                except OSError:
                    pass
                logger.info(
                    "Phase 3 completed. AI confirmed %d semantic play segments.",
                    len(ai_segments),
                )
                # wf294 (all-errored handoff): if EVERY handed-off window failed to yield a
verdict
                # (provider error / clip extraction failed), the run has zero information ?
a FAILURE,
                # not a 0-rally match. A single window that RAN and returned no rallies is a
legit empty
                # and keeps this from firing. (skip_ai has no provider, so it never reaches
here.)
                if handoff_indices and handoff_failed == len(handoff_indices):
                    msg = (
                        f"{label} AI handoff produced no verdict: all {handoff_failed}
candidate "
                        "window(s) errored (provider error / clip extraction failed) ?
incomplete "
                        "run, not a 0-rally match."
                    )
                    logger.error(msg)
                    return Failure(message=msg, is_recoverable=True)

            # --- Phase 4: DB population + candidate linking ---
            segments_data = []
            for segment in ai_segments:
                try:
                    start = float(segment["start_time"])
                    end = float(segment["end_time"])
                    dur = end - start
                    metadata = {
                        "shot_count": self._shot_count_from(segment, dur),
                        "shot_count_confidence": segment.get(
                            "shot_count_confidence", "Unknown"
                        ),
                        "detector": f"{self.family}-hybrid-{model_name}",
                    }
                    segment_id = self.db.add_play_segment(
                        video_id, start, end, None, None, metadata
```

```
518                     )
519                     if segment_id:
```

6. user (2026-07-06T18:00:56.132Z)

```
1       """Lightweight Result type for standardizing success vs. failure across the pipeline.
2
3       This is the start of the Error/Result Contract standardization (low-regret item #3).
4
5       The goal is to make it impossible to confuse "no data / no rallies" with "something
actually failed"
6       (e.g. missing API key, WSL healthcheck failure, provider error, etc.).
7
8       We use simple dataclasses so callers can use isinstance checks or structural pattern
matching.
9       This is intentionally small and incremental ? we start by applying it to the segmenter
10      and provider layers (the biggest sources of silent [] returns today).
11
12      The types are generic where it makes sense. Callers can still access .value on Success
13      or .message on Failure.
14
15      This module has no dependencies on the rest of the app.
16      """
17
18      from __future__ import annotations
19
20      from dataclasses import dataclass, field
21      from typing import Any, Generic, List, Optional, Tuple, TypeVar, Union
22
23      T = TypeVar("T")
24
25
26      @dataclass
27      class Success(Generic[T]):
28          """Represents a successful outcome with an optional list of non-fatal warnings."""
29
30          value: T
31          warnings: List[str] = field(default_factory=list)
32
33          def is_success(self) -> bool:
34              return True
35
36          def unwrap(self) -> T:
37              """Return the value. Only safe to call if you already know it's a Success."""
38              return self.value
39
40
41      @dataclass
42      class Failure:
43          """Represents a failure with a human-readable message and optional details."""
44
45          message: str
46          details: Optional[dict[str, Any]] = None
47          is_recoverable: bool = False  # e.g. missing API key (recoverable) vs internal crash
48
49          def is_success(self) -> bool:
50              return False
51
52          def unwrap(self) -> Any:
53              """Raise to make misuse obvious in code that assumes success."""
54              raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
55
56
57      # Convenient alias for use in type hints: Result[list[dict], Failure]
58      # Result[ValueType] means Success[ValueType] | Failure
```

```
59      # (the Failure "error" type is fixed for this contract)
60      Result = Union[Success[T], Failure]
61
62
63      def unwrap_or_failure(result: Any) -> Tuple[Optional[Any], Optional[Failure]]:
64          """Split a segmenter/provider return into ``(value, failure)`` so a caller branches
once.
65
66          - ``Failure``            -> ``(None, failure)``
67          - ``Success(value=v)``   -> ``(v, None)``
68          - a bare value (legacy code that predates this contract and still returns a raw
list)
69            -> ``(value, None)``
70
71          This is the same isinstance/``.value`` unwrap that ``backend/main.py`` does inline;
callers
72          that forget it (e.g. ``len(process_video(...))``) crash with ``TypeError: ... has no
len()``
73          on a ``Success`` dataclass. ``result`` is typed ``Any`` because the contract is
heterogeneous
74          (Success | Failure | legacy bare value)."""
75          if isinstance(result, Failure):
76              return None, result
77          value = result.value if hasattr(result, "value") else result
78          return value, None
79
```

7. assistant (2026-07-06T18:00:59.565Z)

Now let me look at the yolo_hybrid.py `_run_ai_handoff` and its interaction with `_populate_db`, and the partial test's mocking. I need to check whether patching `_collect_candidate_windows` / `_persist_candidates` actually exercises `_run_ai_handoff` with the two windows.

8. user (2026-07-06T18:01:01.300Z)

```
320             all_candidate_windows: List[Dict[str, Any]],
321             candidate_ids: List[Any],
322             provider,
323             sport_profile,
324             provider_name: str,
325             chunk_duration: float,
326             handoff_padding: float,
327             ai_max_workers: int,
328         ) -> "Union[List[dict], Failure]":
329             """Phase 3: AI Provider Handoff ? re-encode each padded candidate window and ask
330             the provider for semantic boundaries, in parallel.
331
332             Returns the confirmed segments, or a ``Failure`` when EVERY candidate window
failed to
333             yield a verdict (provider error / clip extraction failed) ? an incomplete run
has zero
334             information and must NOT be recorded as a 0-rally "success" (the wf294
class)."""
335             gemini_segments = []
336             gemini_dir = os.path.join(
337                 output_base(self.config), f"chunks_{provider_name}_handoff"
338             )
339             os.makedirs(gemini_dir, exist_ok=True)
340
341             logger.info(
342                 f"Starting Phase 3: AI Provider ({provider_name}) Validation on
{len(all_candidate_windows)} candidate windows..."
343             )
344
345             def process_candidate_window(
```

```
346                 wi: int, window: Dict[str, Any]
347             ) -> Tuple[List[dict], bool]:
348                 """Returns (valid_segments, errored) ? errored=True when this window yielded
NO verdict
349                 (clip extraction failed, or the provider returned metrics['error'])."""
350                 w_start = max(0, window["start"] - handoff_padding)
351                 w_end = min(video_duration, window["end"] + handoff_padding)
352                 w_dur = w_end - w_start
353
354                 w_path = os.path.join(gemini_dir, f"handoff_{video_id}_{wi}.mp4")
355                 # Re-encode short handoff clips to avoid keyframe-boundary drift
356                 success = extract_video_chunk(
357                     video_path, w_start, w_dur, w_path, reencode=True
358                 )
359                 if not success:
360                     return [], True  # extraction failed ? no verdict for this window
361
362                 prompt = sport_profile.get_prompt(chunk_duration=w_dur)
363                 segments, metrics = provider.analyze_video(
364                     w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
365                 )
366                 # B2: a provider failure returns [] with metrics["error"] set ? that is
367                 # "we don't know", NOT "no rallies in this window". Surface it loudly so a
368                 # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
369                 errored = bool(metrics.get("error"))
370                 if errored:
371                     logger.warning(
372                         "[Handoff %d] provider error ? window %d skipped: %s",
373                         wi + 1,
374                         wi,
375                         metrics["error"],
376                     )
377
378                 # Coordinate mapping ? shift local timestamps back to full-video time
379                 valid_segments = []
380                 for segment in segments:
381                     try:
382                         segment["start_time"] = float(segment["start_time"]) + w_start
383                         segment["end_time"] = float(segment["end_time"]) + w_start
384                         # Stamp the source candidate so Phase 4 can link
prediction->confirmation.
385                         segment["_candidate_id"] = candidate_ids[wi]
386                         valid_segments.append(segment)
387                     except (ValueError, TypeError, KeyError) as e:
388                         logger.warning(
389                             f"Dropping segment with unparseable timestamps: {segment} ? {e}"
390                         )
391
392                 try:
393                     os.remove(w_path)
394                 except OSError:
395                     pass
396
397                 return valid_segments, errored
398
399             with concurrent.futures.ThreadPoolExecutor(
400                 max_workers=ai_max_workers
401             ) as executor:
402                 futures = [
403                     executor.submit(process_candidate_window, wi, w)
404                     for wi, w in enumerate(all_candidate_windows)
405                 ]
406                 # Iterate in submission order to preserve temporal ordering
407                 window_results = [future.result() for future in futures]
```

```
            for valid, _errored in window_results:
                gemini_segments.extend(valid)
            handoff_failed = sum(1 for _valid, errored in window_results if errored)

            try:
                os.rmdir(gemini_dir)
            except OSError:
                pass

            logger.info(
                f"Phase 3 completed. AI confirmed {len(gemini_segments)} semantic play
segments."
            )
            # wf294 (all-errored handoff): if EVERY candidate window failed to yield a
verdict
            # (provider error / clip extraction failed), the run has zero information ? a
FAILURE, not
            # a 0-rally match. A window that RAN and returned no rallies keeps this from
firing.
            if all_candidate_windows and handoff_failed == len(all_candidate_windows):
                msg = (
                    f"yolo_hybrid AI handoff produced no verdict: all {handoff_failed}
candidate "
                    "window(s) errored (provider error / clip extraction failed) ?
incomplete run, "
                    "not a 0-rally match."
                )
                logger.error(msg)
                return Failure(message=msg, is_recoverable=True)
            return gemini_segments

        def _populate_db(
            self, video_id: str, gemini_segments: List[dict], model_name: str
        ) -> List[Dict[str, Any]]:
            """Phase 4: DB Population ? write confirmed AI segments, link them back to their
            source candidate, and build the returned segments_data. Verbatim move."""
            segments_data = []
            for segment in gemini_segments:
                try:
                    start = float(segment["start_time"])
                    end = float(segment["end_time"])
                    dur = end - start

                    shot_count = segment.get("shuttle_exchange_count")
                    if shot_count is None:
                        shot_count = max(3, int(dur / 0.8))
                    else:
                        try:
                            shot_count = int(shot_count)
                        except (ValueError, TypeError):
                            shot_count = max(3, int(dur / 0.8))

                    metadata = {
                        "shot_count": shot_count,
                        "shot_count_confidence": segment.get(
                            "shot_count_confidence", "Unknown"
                        ),
                        "detector": f"yolo-hybrid-{model_name}",
                    }

                    segment_id = self.db.add_play_segment(
                        video_id, start, end, None, None, metadata
                    )
                    if segment_id:
                        # Link back to the candidate that produced this segment. A candidate
```

```
                        # may yield multiple AI segments (one-to-many); confirmed_segment_id
keeps
                        # the last one ? the link table records "this candidate was
confirmed".
                        candidate_id = segment.get("_candidate_id")
                        if candidate_id is not None:
                            self.db.link_candidate_to_segment(candidate_id, segment_id)
                        segments_data.append(
                            {
                                "id": segment_id,
                                "start_time": start,
                                "end_time": end,
                                "duration": dur,
                                "metadata": metadata,
                            }
                        )
                except Exception as e:
                    logger.warning(f"Skipping invalid segment struct: {e}")

            return segments_data

        def process_video(  # type: ignore[override]
            self,
            video_id: str,
            video_path: str,
            player_pool: List[str] = None,
            max_frames: int = None,
        ) -> "Union[List[Dict[str, Any]], Failure]":
            # Result contract: an unrunnable config (bad sport/provider, missing key) or an
unreadable
            # video returns a Failure so the run is marked failed ? NOT a silent 0-rally
"success"
            # (the wf294 class). A genuine empty match still returns a bare []. Every caller
branches
            # on isinstance(result, Failure).
            self.person.lazy_load_model()

            # Phase 0: resolve sport profile + AI provider (returns a Failure on any config
failure).
            provider = self._resolve_provider(video_path)
            if isinstance(provider, Failure):
                return provider

            # Re-read the config values needed downstream (pure deterministic reads).
            idx_cfg = self.config.indexing
            sport_profile = sport_registry.get(self.config.sport)
            provider_name = self.config.ai_provider
            model_name = self.config.ai_model

            video_duration = get_video_duration(video_path)
            if video_duration <= 0:
                logger.error("Invalid video duration.")
                # An unreadable/broken video, not a 0-rally match.
                return Failure(
                    message=f"yolo_hybrid: invalid/unreadable video duration
({video_duration})."
                )

            chunk_duration = idx_cfg.chunk_duration
            chunk_overlap = idx_cfg.chunk_overlap
            velocity_thresh = (
                idx_cfg.yolo_velocity_threshold
            )  # Fraction of frame width per second
            merge_gap = idx_cfg.dead_time_merge_gap
            frame_skip = idx_cfg.yolo_frame_skip
```

```
            tracker_config = idx_cfg.yolo_tracker
            handoff_padding = idx_cfg.ai_handoff_padding
            ai_max_workers = idx_cfg.ai_max_workers
            log_candidates = idx_cfg.log_yolo_candidates
            store_candidates = idx_cfg.store_yolo_candidates

            # Snapshot of the params that produced these detections ? stored alongside each
            # candidate so a fine-tuning run can reproduce / correlate against them.
            detection_params = {
                "velocity_thresh": velocity_thresh,
                "merge_gap": merge_gap,
                "frame_skip": frame_skip,
                "tracker": tracker_config,
                "detector_model": idx_cfg.detector_model,
                "detector_score_threshold": idx_cfg.detector_score_threshold,
                "min_action_window_duration": idx_cfg.min_action_window_duration,
            }

            chunks_dir = os.path.join(output_base(self.config), "chunks_yolo")
            os.makedirs(chunks_dir, exist_ok=True)

            # Phase 2: chunk ? per-chunk action windows ? cross-chunk overlap merge.
            all_candidate_windows = self._collect_candidate_windows(
                video_id,
                video_path,
                video_duration,
                idx_cfg,
                chunk_duration,
                chunk_overlap,
                velocity_thresh,
                merge_gap,
                frame_skip,
                chunks_dir,
            )

            # Phase 2b: log + persist raw candidates (window index -> candidate_id).
            candidate_ids = self._persist_candidates(
                video_id,
                all_candidate_windows,
                detection_params,
                log_candidates,
                store_candidates,
            )

            try:
                os.rmdir(chunks_dir)
            except Exception:
                pass

            # Phase 3: AI Provider Handoff (returns a Failure when every window errored ?
wf294).
            handoff = self._run_ai_handoff(
                video_id,
                video_path,
                video_duration,
                all_candidate_windows,
                candidate_ids,
                provider,
                sport_profile,
                provider_name,
                chunk_duration,
                handoff_padding,
                ai_max_workers,
            )
            if isinstance(handoff, Failure):
```

```
589                 return handoff
590
591             # Phase 4: DB Population
592             return self._populate_db(video_id, handoff, model_name)
593
594
595     # Register the segmenter
596     segmenter_registry.register(HybridYoloSegmenter)
597
```

9. assistant (2026-07-06T18:01:06.435Z)

The yolo partial test patches `_collect_candidate_windows` to return two windows and `_persist_candidates` to return `[None, None]`. Then the real `_run_ai_handoff` runs with the actual `provider` (mocked) and `extract_video_chunk` mocked to True. That genuinely exercises the new logic. Note it does NOT patch `_run_ai_handoff` ? good.

Now let me check a critical concern: in the yolo partial test, `candidate_ids = [None, None]`. In `process_candidate_window`, on window 0 it stamps `segment["_candidate_id"] = candidate_ids[0]` = None. That's fine. Let me check the worker `_fail` path and the caller contracts. Let me look at `cmd_infer` and the callers.

10. user (2026-07-06T18:01:07.580Z)

```
270             local_video,
271             "failed" if failures else "processed",
272             [r.model_dump() for r in val_results],
273         )
274
275         def _fail(rc: int) -> int:
276             # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
277             # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
278             # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
279             # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
280             # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
281             # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
282             # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
283             try:
284                 _serialize_and_push_outputs(
285                     scratch,
286                     args.out_uri,
287                     video_id,
288                     [],
289                     "failed",
290                     storage_cfg,
291                     str(getattr(args, "video_uri", "") or ""),
292                 )
293             except Exception as e:  # never mask the real failure on an artifact-push hiccup
294                 logger.warning("[worker] could not push failure artifacts: %s", e)
295             # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
296             if getattr(args, "done_uri", None):
297                 _write_done_marker(
298                     args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
299                 )
300             return rc
301
302         segments: List[dict] = []
303         segmenter_actual = None
```

```
        segmentation_ran = False
        status = "failed"
        try:
            if failures:
                print(
                    "[worker] validation FAILED: "
                    + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
                )
                return _fail(2)
            seg_name = args.segmenter or config.indexing.default_segmenter
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(
                    f"[worker] unknown segmenter: {seg_name!r} "
                    f"(have {list(segmenter_registry.list_available())})"
                )
                return _fail(2)
            segmenter_actual = seg_name
            result = segmenter_cls(db, config).process_video(
                video_id, local_video, max_frames=args.max_frames
            )
            segmentation_ran = True
            if isinstance(result, Failure):
                print(f"[worker] segmenter FAILED: {result.message}")
                return _fail(3)
            segments = result.value if hasattr(result, "value") else result
            status = "success"
            # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
            # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
            # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
            if not segments:
                detector_impl = (
                    os.environ.get("RALLY_DETECTOR_IMPL")
                    or getattr(config.indexing, "detector_impl", None)
                    or "auto"
                )
                logger.warning(
                    "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
                    "The detector substrate yielded no usable trajectories/windows ? check
the detector "
                    "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
                    "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
                    "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
                    "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
                    "detections, or the input resolution differs from the tuned proxy
resolution. "
                    "Re-run with -v for the native command + frame counts.",
                    video_id,
                    segmenter_actual,
                    detector_impl,
                )
        finally:
            # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
            emit_indexer_report(
                db=db,
                video_id=video_id,
                video_path=local_video,
```

```
359                 config=config,
360                 val_results=val_results,
361                 val_context=val_context,
362                 segmenter_requested=args.segmenter,
363                 segmenter_actual=segmenter_actual,
364                 was_reingest=False,
365                 segmentation_ran=segmentation_ran,
366             )
367
368         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
369         pushed = _serialize_and_push_outputs(
370             scratch,
371             args.out_uri,
372             video_id,
373             segments,
374             status,
375             storage_cfg,
376             str(getattr(args, "video_uri", "") or ""),
377         )
378
379         print(
380             f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
381         )
382         for u in pushed:
383             print("   ", u)
384
385         # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
386         # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
387         # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
388         # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
389         if getattr(args, "done_uri", None):
```

11. assistant (2026-07-06T18:01:16.739Z)

The worker path is coherent. `_fail` handles rc 2 (validation/unknown segmenter) and rc 3 (segmenter Failure), both writing status="failed". Now let me verify the callers `main.py:1557` and `orchestration.py:806` branch correctly, and check whether `_run_ai_handoff` in yolo returns the `Failure` even in the `skip_ai` case (yolo does not have a skip_ai branch in the diff). Let me check whether yolo_hybrid has a skip_ai path at all ? the contract note says "skip_ai (fully-local) has no provider and never reaches this path". Let me verify yolo's `_resolve_provider` under skip_ai.

12. user (2026-07-06T18:01:18.316Z)

```
63                 "candidate filtering, then an AI provider for high-precision semantic
boundaries."
64             )
65
66         def _resolve_provider(self, video_path: str):
67             """Phase 0: resolve sport profile + AI provider (with API key).
68
69             Returns the configured provider instance, or a ``Failure`` (bad sport, missing
API key,
70             or unknown provider) that ``process_video`` propagates ? so an unrunnable config
is marked
71             FAILED with a reason instead of a silent 0-rally "success" (the wf294 loud-
failures bar;
72             mirrors the gemini/trajectory segmenters). A genuine "ran, found nothing" is
unaffected.
73             """
```

```
74              # Get sport profile config
75              sport_name = self.config.sport
76              try:
77                  sport_registry.get(sport_name)
78              except KeyError as e:
79                  logger.error(str(e))
80                  return Failure(str(e))
81
82              # Get AI provider and model config
83              provider_name = self.config.ai_provider
84              model_name = self.config.ai_model
85
86              # Get appropriate API key based on the provider
87              api_key_env_var = f"{provider_name.upper()}_API_KEY"
88              api_key = os.environ.get(api_key_env_var)
89              if not api_key:
90                  from backend.pipeline.segmenters._keyhint import api_key_hint
91
92                  msg = (
93                      f"{api_key_env_var} environment variable is not set! "
94                      f"To run the {provider_name} segmenter, set your API key. "
95                      + api_key_hint(api_key_env_var)
96                  )
97                  logger.error(msg)
98                  return Failure(msg, is_recoverable=True)
99
100             try:
101                 provider = provider_registry.get(
102                     provider_name, api_key=api_key, model_name=model_name
103                 )
104             except KeyError as e:
105                 logger.error(str(e))
106                 return Failure(str(e))
107
108             logger.info(
109                 f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}"
110             )
111             return provider
112
113         def _collect_candidate_windows(
114             self,
115             video_id: str,
116             video_path: str,
117             video_duration: float,
118             idx_cfg: Dict[str, Any],
119             chunk_duration: float,
120             chunk_overlap: float,
121             velocity_thresh: float,
122             merge_gap: float,
123             frame_skip: int,
124             chunks_dir: str,
125         ) -> List[Dict[str, Any]]:
126             """Phase 2: chunk the video, extract per-chunk action windows (pipelined or
127             sequential), then dedup-merge overlapping windows across chunks. Verbatim
move."""
128             step = chunk_duration - chunk_overlap
129             chunk_starts = []
130             t = 0.0
131             while t < video_duration:
132                 chunk_starts.append(t)
133                 t += step
134
135             num_chunks = len(chunk_starts)
136             logger.info(
```

```
137                 f"Video duration: {video_duration:.1f}s. Splitting into {num_chunks}
chunks."
138             )
139
140             all_candidate_windows = []
141
142             t_start = time.time()
143             prefetch_workers = idx_cfg.yolo_prefetch_workers
144
145             if num_chunks > 1 and prefetch_workers > 0:
146                 # --- PIPELINED PROCESSING ---
147                 # GPU inference must stay sequential (single GPU), but ffmpeg chunk
148                 # extraction is pure I/O. We pre-extract upcoming chunks in background
149                 # threads so extraction of chunk N+1 overlaps with inference on chunk N.
150                 logger.info(
151                     f"Running detection Phase 2 with pipelined extraction
({prefetch_workers} prefetch workers)..."
152                 )
153
154                 extract_executor = concurrent.futures.ThreadPoolExecutor(
155                     max_workers=prefetch_workers
156                 )
157
158                 def _extract_chunk(ci: int, cs: float) -> Tuple[int, float, str, bool]:
159                     chunk_end = min(cs + chunk_duration, video_duration)
160                     actual_dur = chunk_end - cs
161                     chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
162                     success = extract_video_chunk(video_path, cs, actual_dur, chunk_path)
163                     return (ci, cs, chunk_path, success)
164
165                 # Submit all extractions upfront ? they'll queue and run as workers free up
166                 extract_futures = [
167                     extract_executor.submit(_extract_chunk, ci, cs)
168                     for ci, cs in enumerate(chunk_starts)
169                 ]
170
171                 # Process results in order: wait for extraction, then run inference on GPU
172                 for future in extract_futures:
173                     ci, chunk_start, chunk_path, success = future.result()
174                     chunk_end = min(chunk_start + chunk_duration, video_duration)
175                     logger.info(
176                         f"Processing Chunk {ci + 1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)..."
177                     )
178
179                     if not success:
180                         logger.warning(f"Chunk {ci + 1} extraction failed. Skipping.")
181                         continue
182
```

13. assistant (2026-07-06T18:01:25.194Z)

Note: yolo_hybrid has no `skip_ai` branch ? it always resolves a provider. Under `skip_ai_handoff=True`, `_resolve_provider` still requires an API key and would return a Failure. But that is pre-existing behavior (yolo always requires the key), not introduced by this change, and the contract note explicitly says "skip_ai (fully-local) has no provider" applies to the trajectory family. So no defect there.

Now let me check a subtle F3 concern: in yolo, when `all_candidate_windows` is empty, `_run_ai_handoff` returns `gemini_segments` (empty list) ? the `all_candidate_windows and ...` guard prevents firing. Good. But there's a question about whether yolo reaches `_run_ai_handoff` at all with zero windows ? it does (no early `if not all_candidate_windows: return []`), which means with zero windows it returns `[]`. Fine.

Let me now verify the callers in main.py and orchestration.py to confirm the contract, and check

the gemini/motion segmenters for the convention baseline (is_recoverable usage).

14. user (2026-07-06T18:01:26.803Z)

```
1545                        print(f"[FAIL] Segmenter '{seg_name}' not found.")
1546                        return
1547
1548                    print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
1549                    segmenter_actual = seg_name
1550                    play_wm, cand_wm = db_instance.segment_watermarks(video_id)
1551                    segmenter = segmenter_cls(db_instance, global_config)
1552                    result = segmenter.process_video(
1553                        video_id, local_video, max_frames=args.max_frames
1554                    )
1555                    segmentation_ran = True
1556
1557                    if isinstance(result, Failure):
1558                        print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
1559                        print("[FAIL] Indexing halted for video.")
1560                        segments = []
1561                    else:
1562                        segments = result.value if hasattr(result, "value") else result
1563                        print(
1564                            f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}"
1565                        )
1566                    # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
1567                    _finalize_reingest(db_instance, video_id, segments, play_wm, cand_wm)
1568                finally:
1569                    # Always emit the per-video observability report. Never raises.
1570                    paths = emit_indexer_report(
1571                        db=db_instance,
1572                        video_id=video_id,
1573                        # Resolved LOCAL path, not the raw URI (see #397) ? a remote URI
would write a
1574                        # literal scheme dir (gcs:/, gs:/, local:/) at the repo root. Source
URI is kept
1575                        # in DB provenance via db.add_video.
1576                        video_path=local_video,
1577                        config=_report_config_for_input(global_config, args.video_path),
1578                        val_results=val_results,
1579                        val_context=val_context,
1580                        segmenter_requested=args.segmenter,
1581                        segmenter_actual=segmenter_actual,
1582                        was_reingest=was_reingest,
1583                        segmentation_ran=segmentation_ran,
1584                    )
```

15. user (2026-07-06T18:01:28.031Z)

```
File does not exist. Note: your current working directory is C:\Users\avidu\Projects\khelsutra-
guru\badminton-highlight-indexer.
```

16. assistant (2026-07-06T18:01:30.706Z)

Let me find orchestration.py and check the caller there, plus look at the gemini/motion segmenters for the convention baseline on is_recoverable.

17. user (2026-07-06T18:01:31.340Z)

```
backend\orchestration.py
```

18. user (2026-07-06T18:01:32.405Z)

```
tests\test_base.py:21:            return Failure(message="Simulated failure for test",
is_recoverable=True)
tests\test_base.py:47:    assert failure.is_recoverable is True
tests\test_base.py:146:    failure = Failure(message="segmenter exploded", is_recoverable=False)
backend\results.py:47:    is_recoverable: bool = False  # e.g. missing API key (recoverable) vs
internal crash
tests\test_motion_segmenter.py:111:    assert result.is_recoverable is False
backend\pipeline\segmenters\motion.py:40:                message=f"Could not open video:
{video_path}", is_recoverable=False
backend\pipeline\segmenters\trajectory_hybrid.py:225:                return Failure(msg,
is_recoverable=True)
backend\pipeline\segmenters\trajectory_hybrid.py:290:                is_recoverable=True,
backend\pipeline\segmenters\trajectory_hybrid.py:311:                is_recoverable=True,
backend\pipeline\segmenters\trajectory_hybrid.py:500:                return Failure(message=msg,
is_recoverable=True)
backend\pipeline\segmenters\yolo_hybrid.py:98:            return Failure(msg,
is_recoverable=True)
backend\pipeline\segmenters\yolo_hybrid.py:430:            return Failure(message=msg,
is_recoverable=True)
tests\test_results.py:16:    f = Failure(message="Could not open video: x.mp4",
is_recoverable=False)
tests\test_tracknet_hybrid.py:111:    assert isinstance(res, Failure) and res.is_recoverable
tests\test_tracknet_hybrid.py:159:    assert "no trajectory" in res.message and
res.is_recoverable
tests\test_trajectory_hybrid_equivalence.py:145:    assert isinstance(res, Failure) and
res.is_recoverable  # all (1) windows errored -> no verdict
tests\test_worker.py:214:            is_recoverable=True,
tests\test_yolo_hybrid.py:153:    assert isinstance(res, Failure) and res.is_recoverable  # all
(1) windows errored -> no verdict
tests\test_yolo_hybrid.py:215:    assert isinstance(res, Failure) and res.is_recoverable
```

19. assistant (2026-07-06T18:01:38.766Z)

Now let me check the gemini segmenter (the stated convention baseline) and orchestration.py:806 caller, plus check whether `is_recoverable` is actually *read* by any consumer (the review asks this specifically).

20. user (2026-07-06T18:01:40.844Z)

```
806:        if isinstance(result, Failure):
807:            logger.error(f"[API] Segmenter failed: {result.message}")
812:                "message": f"Segmenter '{seg_name}' failed: {result.message}",
```

21. user (2026-07-06T18:01:42.145Z)

```
100:            return Failure(str(e))
116:            return Failure(msg)
124:            return Failure(str(e))
272:            # from "genuinely no rallies in this chunk" (both return [] today).
693:            return []
```

22. assistant (2026-07-06T18:01:45.567Z)

Interesting. The gemini segmenter (the stated convention baseline) uses `Failure(msg)` for the **missing API key** case (line 116) ? WITHOUT `is_recoverable=True`. Let me verify this precisely, since the diff's trajectory/yolo changes set `is_recoverable=True` for missing-key and the comments claim "mirrors gemini".

23. user (2026-07-06T18:01:46.367Z)

```
90              with a reason, instead of a silent 0-rally "success" that looks like an empty
match
91              (Finding B ? the loud-failures bar). A genuine "ran, found nothing" is
unaffected.
```

```
92              """
93              # Get sport profile config (config is always typed AppConfig ?
_normalize_config()
94              # in base.py converts plain dicts passed by tests at init time)
95              sport_name = self.config.sport
96              try:
97                  sport_profile = sport_registry.get(sport_name)
98              except KeyError as e:
99                  logger.error(str(e))
100                 return Failure(str(e))
101
102             # Get AI provider and model config
103             provider_name = self.config.ai_provider
104
105             # Get appropriate API key based on the provider
106             api_key_env_var = f"{provider_name.upper()}_API_KEY"
107             api_key = os.environ.get(api_key_env_var)
108             if not api_key:
109                 from backend.pipeline.segmenters._keyhint import api_key_hint
110
111                 msg = (
112                     f"{api_key_env_var} is not set ? the {provider_name} segmenter cannot
run. "
113                     f"Set your API key. " + api_key_hint(api_key_env_var)
114                 )
115                 logger.error(msg)
116                 return Failure(msg)
117
118             try:
119                 provider = provider_registry.get(
120                     provider_name, api_key=api_key, model_name=model_name
121                 )
122             except KeyError as e:
123                 logger.error(str(e))
124                 return Failure(str(e))
125
126             return sport_profile, provider
127
128         def _maybe_trim_video(
129             self, video_id: str, video_path: str
```

24. assistant (2026-07-06T18:01:53.392Z)

This is a real convention inconsistency. The gemini segmenter (which the diff's comments repeatedly cite as the baseline: "mirrors the gemini segmenter", "mirrors gemini") returns `Failure(msg)` for the missing-API-key path with the **default** `is_recoverable=False`. The new trajectory_hybrid (line 225) and yolo_hybrid (line 98) set `is_recoverable=True` for the same missing-key case, with a comment explicitly claiming "mirrors gemini". This is a divergence from the cited baseline.

However, I need to assess whether this is a *defect* or a defensible choice. Let me check whether `is_recoverable` is read by ANY consumer ? the review explicitly asks this. If no consumer reads it, then the inconsistency is cosmetic/latent (not a behavior bug), but it's still a real convention inconsistency where the comment ("mirrors gemini") is factually wrong. Let me grep all reads of `.is_recoverable` in non-test backend code.

25. user (2026-07-06T18:01:54.756Z)

No matches found

26. assistant (2026-07-06T18:02:03.807Z)

Confirmed: no backend consumer reads `.is_recoverable`. So the `is_recoverable=True` vs gemini's `is_recoverable=False` divergence for the missing-key path has **no behavioral effect today** ?

it's a latent convention inconsistency where the comments' claim "mirrors gemini" is factually incorrect. This is a real but minor/low finding (a stale/incorrect comment + inconsistent metadata that a future consumer of `is_recoverable` could act on differently for the same failure class across three segmenters).

Now let me focus on the more important test-quality questions from the review. Let me check the trajectory equivalence test's `_runner` helper and whether the partial test actually goes through the handoff correctly with two windows, and verify the `_select_for_handoff` behavior doesn't filter out window 1. Let me read the full equivalence test file.

27. user (2026-07-06T18:02:04.365Z)

```
1       """Equivalence contract for the trajectory-hybrid twins.
2
3       tracknet_hybrid and wasb_hybrid were copy-paste twins that drifted (the
4       shot_count parse); both are now thin subclasses of TrajectoryHybridSegmenter.
5       These tests run BOTH through the same scenarios and assert identical P3/P4
6       behavior, so any future divergence has to be introduced deliberately, in the
7       base, for both at once.
8       """
9
10      from unittest.mock import MagicMock, patch
11
12      import pytest
13
14      from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
15      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
16      from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
17      from backend.results import Failure
18
19      TWINS = [
20          (TrackNetHybridSegmenter, "tracknet", "tracknet_hybrid"),
21          (WasbHybridSegmenter, "wasb", "wasb_hybrid"),
22      ]
23
24
25      def _window(start=1.0, end=4.0):
26          return {
27              "start": start,
28              "end": end,
29              "active_frame_count": 30,
30              "peak_velocity": 0.4,
31              "mean_velocity": 0.2,
32              "track_id_count": 1,
33              "chunk_index": 0,
34          }
35
36
37      def _runner(windows=None):
38          runner = MagicMock()
39          runner.healthcheck.return_value = (True, "env ok")
40          runner.run_predict.return_value = "out.csv"
41          runner.parse_trajectory_csv.return_value = [MagicMock(visible=True)]
42          runner.trajectory_to_action_windows.return_value = (
43              windows if windows is not None else [_window()]
44          )
45          return runner
46
47
48      def _run(cls, provider_segments, db=None):
49          """Run one segmenter class through the standard mocked pipeline."""
50          provider = MagicMock()
51          provider.analyze_video.return_value = (provider_segments, {})
52          db = db or MagicMock()
53          db.add_candidate_segment.return_value = 55
```

```
        db.add_play_segment.return_value = 200

        with patch.object(
            cls, "_build_runner", return_value=(_runner(), {"weights_path": "w"})
        ), patch.object(cls, "_probe_fps_width", return_value=(30.0, 1280.0)), patch(
            "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
            return_value=30.0,
        ), patch(
            "backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
            return_value=True,
        ), patch(
            "backend.pipeline.segmenters.trajectory_hybrid.provider_registry"
        ) as preg, patch("os.environ.get", return_value="dummy_key"):
            preg.get.return_value = provider
            seg = cls(db, {"indexing": {}})
            return db, seg.process_video("v1", "clip.mp4")


    @pytest.mark.parametrize("cls,family,detector", TWINS)
    def test_detector_identity_flows_into_db_rows(cls, family, detector):
        db, res = _run(
            cls, [{"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count": 4}]
        )
        assert len(res) == 1
        _, kwargs = db.add_candidate_segment.call_args
        assert kwargs["detector"] == detector
        assert kwargs["detection_params"]["detector"] == detector
        # P4 metadata tags the family + model.
        assert res[0]["metadata"]["detector"].startswith(f"{family}-hybrid-")
        db.link_candidate_to_segment.assert_called_once_with(55, 200)


    @pytest.mark.parametrize("cls,family,detector", TWINS)
    @pytest.mark.parametrize(
        "exchange,expected",
        [
            (4, 4),  # provider-reported count wins
            ("7", 7),  # numeric strings coerce
            (None, 3),  # explicit null -> duration estimate (2.5s/0.8 -> 3 floor)
            ("garbage", 3),  # unparseable -> same estimate
        ],
    )
    def test_shot_count_behavior_is_identical(cls, family, detector, exchange, expected):
        """The historical drift point: wasb_hybrid relied on int(None) raising into
        the fallback while tracknet_hybrid short-circuited. One implementation now."""
        seg_dict = {"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count": exchange}
        _, res = _run(cls, [seg_dict])
        assert res[0]["metadata"]["shot_count"] == expected


    @pytest.mark.parametrize("cls,family,detector", TWINS)
    def test_provider_offsets_applied_identically(cls, family, detector):
        # Window [1,4] + 2.0s default padding -> clip starts at max(0, -1) = 0.0;
        # provider-relative 0.5 stays 0.5 absolute here.
        _, res = _run(cls, [{"start_time": 0.5, "end_time": 3.0}])
        assert res[0]["start_time"] == 0.5
        assert res[0]["end_time"] == 3.0


    @pytest.mark.parametrize("cls,family,detector", TWINS)
    def test_provider_failure_is_surfaced_not_silent(cls, family, detector, caplog):
        """B2/P17 + F3 (wf294): a provider failure ([], metrics['error']) must be logged
loudly, not
        silently treated as 'no rallies'. When it errors on the ONLY candidate window the
whole run has
```

```
        zero verdict, so process_video now returns a Failure (was a bare [] that looked like
a 0-rally
        match) ? the worker marks it failed/retryable instead of a silent empty
'success'."""
        import logging

        provider = MagicMock()
        provider.analyze_video.return_value = (
            [],
            {"error": "generate failed after retries: 503", "generate_failed": True},
        )
        db = MagicMock()
        db.add_candidate_segment.return_value = 55
        with patch.object(
            cls, "_build_runner", return_value=(_runner(), {"weights_path": "w"})
        ), patch.object(cls, "_probe_fps_width", return_value=(30.0, 1280.0)), patch(
            "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
            return_value=30.0,
        ), patch(
            "backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
            return_value=True,
        ), patch(
            "backend.pipeline.segmenters.trajectory_hybrid.provider_registry"
        ) as preg, patch("os.environ.get", return_value="dummy_key"):
            preg.get.return_value = provider
            seg = cls(db, {"indexing": {}})
            with caplog.at_level(
                logging.WARNING, logger="backend.pipeline.segmenters.trajectory_hybrid"
            ):
                res = seg.process_video("v1", "clip.mp4")
        assert isinstance(res, Failure) and res.is_recoverable  # all (1) windows errored ->
no verdict
        assert any(
            "provider error" in r.message and "generate failed" in r.message
            for r in caplog.records
        ), [r.message for r in caplog.records]


    @pytest.mark.parametrize("cls,family,detector", TWINS)
    def test_partial_handoff_error_is_a_success_not_a_failure(cls, family, detector):
        """The boundary of F3: when SOME candidate windows error but at least one RAN (even
if it found
        no rallies), the run is NOT a failure ? only an all-errored handoff is. Here two
windows are
        handed off; the first returns a rally, the second errors -> a Success with the first
rally."""
        provider = MagicMock()
        provider.analyze_video.side_effect = [
            ([{"start_time": 0.5, "end_time": 2.0}], {}),  # window 0: a real verdict
            ([], {"error": "generate failed after retries: 503"}),  # window 1: errored
        ]
        db = MagicMock()
        db.add_candidate_segment.side_effect = [55, 56]
        db.add_play_segment.return_value = 200
        two_windows = [_window(1.0, 4.0), _window(10.0, 13.0)]
        with patch.object(
            cls, "_build_runner", return_value=(_runner(two_windows), {"weights_path": "w"})
        ), patch.object(cls, "_probe_fps_width", return_value=(30.0, 1280.0)), patch(
            "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
            return_value=30.0,
        ), patch(
            "backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
            return_value=True,
        ), patch(
            "backend.pipeline.segmenters.trajectory_hybrid.provider_registry"
```

```
176         ) as preg, patch("os.environ.get", return_value="dummy_key"):
177             preg.get.return_value = provider
178             seg = cls(db, {"indexing": {}})
179             res = seg.process_video("v1", "clip.mp4")
180         assert not isinstance(res, Failure)
181         assert len(res) == 1 and res[0]["start_time"] == 0.5
182 
183 
184     def test_base_is_not_registered():
185         from backend.pipeline.segmenters.base import segmenter_registry
186 
187         available = segmenter_registry.list_available()
188         assert "tracknet_hybrid" in available
189         assert "wasb_hybrid" in available
190         # The base engine itself must not appear as a selectable segmenter.
191         assert not any(
192             isinstance(cls, type) and cls is TrajectoryHybridSegmenter
193             for cls in available.values()
194         )
195 
196 
197     @pytest.mark.parametrize("cls,family,detector", TWINS)
198     def test_persisted_stats_are_exactly_the_four_motion_keys(cls, family, detector):
199         """The fusion-P2 enrichment seam must leave the twins' persisted stats byte-
identical:
200         the base _enrich_candidate_stats is identity, so NO net_crossings / person keys leak
in."""
201         db, _res = _run(cls, [{"start_time": 0.5, "end_time": 3.0}])
202         _, kwargs = db.add_candidate_segment.call_args
203         assert set(kwargs["stats"]) == {
204             "peak_velocity",
205             "mean_velocity",
206             "active_frame_count",
207             "track_id_count",
208         }
209 
210 
211     @pytest.mark.parametrize("cls,family,detector", TWINS)
212     def test_base_fusion_hooks_are_identity(cls, family, detector):
213         """Directly pin the two fusion seams on the base: stats = the 4 motion keys
verbatim,
214         and every window is selected for handoff (no gating)."""
215         seg = cls(MagicMock(), {"indexing": {}})
216         cw = _window()
217         stats = seg._enrich_candidate_stats(cw, [], 30.0, 1280.0, "v.mp4", {})
218         assert stats == {
219             "peak_velocity": 0.4,
220             "mean_velocity": 0.2,
221             "active_frame_count": 30,
222             "track_id_count": 1,
223         }
224         assert seg._select_for_handoff([cw, cw, cw], [stats, stats, stats], {}) == [0, 1, 2]
225 
```

28. assistant (2026-07-06T18:02:11.787Z)

The trajectory F3 tests are well-formed: both the all-errored (line 145: `isinstance(res, Failure) and res.is_recoverable`) and the partial-success (line 180-181: `not isinstance Failure` + real segment) are parametrized across both twins. `_select_for_handoff` is confirmed identity (line 224), so both windows in the partial test actually reach the handoff. Good.

Now the critical question the review flagged: does the yolo partial test's mocking of `_collect_candidate_windows`/`_persist_candidates` bypass so much that it doesn't exercise `_run_ai_handoff`'s new logic? Let me re-examine. The test patches those two methods but NOT `_run_ai_handoff`, and `extract_video_chunk` is mocked True and

`provider.analyze_video.side_effect` gives window 0 a verdict and window 1 an error. So `_run_ai_handoff` genuinely runs with 2 windows, `handoff_failed=1 != 2`, returns `gemini_segments` with 1 seg. Then `_populate_db` runs. That is a sound exercise of the new logic. Good ? no defect there.

Now let me check the yolo test file fully to see if there is an all-errored yolo test AND whether the missing-key/other converted paths are covered.

29. user (2026-07-06T18:02:13.256Z)

```
100         assert len(res) == 1
101         assert res[0]["id"] == "yolo_segment_1"
102         assert res[0]["start_time"] == 0.0
103         assert res[0]["end_time"] == 3.0
104
105
106     @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
107     @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
108     @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
109     @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
110     @patch("os.environ.get")
111     def test_yolo_hybrid_provider_failure_is_surfaced_not_silent(
112         mock_env_get, mock_provider_registry, mock_extract, mock_get_dur, mock_cap, caplog
113     ):
114         """B2/P17 + F3 (wf294): a provider failure ([], metrics['error']) must be logged
loudly, not
115         silently treated as 'no rallies'. When it errors on the ONLY candidate window the
whole run has
116         zero verdict, so process_video now returns a Failure (was a bare [] that looked like
a 0-rally
117         match) ? the worker marks it failed/retryable instead of a silent empty
'success'."""
118         import logging
119
120         mock_env_get.return_value = "dummy_api_key"
121         mock_get_dur.return_value = 10.0
122         mock_extract.return_value = True
123
124         mock_provider = MagicMock()
125         mock_provider.analyze_video.return_value = (
126             [],
127             {"error": "generate failed after retries: 503", "generate_failed": True},
128         )
129         mock_provider_registry.get.return_value = mock_provider
130
131         db_mock = MagicMock()
132         db_mock.add_play_segment.return_value = "yolo_segment_1"
133         config = {
134             "indexing": {
135                 "use_gpu": False,
136                 "chunk_duration": 100,
137                 "chunk_overlap": 10,
138                 "yolo_velocity_threshold": 0.1,
139                 "dead_time_merge_gap": 2.0,
140                 "yolo_frame_skip": 1,
141                 "min_action_window_duration": 1.0,
142             }
143         }
144
145         segmenter = HybridYoloSegmenter(db_mock, config)
146         _mock_stage1(segmenter)
147         mock_cap.return_value = _make_cap_mock()
148
149         with caplog.at_level(
150             logging.WARNING, logger="backend.pipeline.segmenters.yolo_hybrid"
```

```
        ):
            res = segmenter.process_video("v1", "dummy.mp4")
        assert isinstance(res, Failure) and res.is_recoverable  # all (1) windows errored ->
no verdict
        assert any(
            "provider error" in r.message and "generate failed" in r.message
            for r in caplog.records
        ), [r.message for r in caplog.records]


    @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
    @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
    @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
    @patch("os.environ.get")
    def test_yolo_hybrid_partial_handoff_error_is_a_success_not_a_failure(
        mock_env_get, mock_provider_registry, mock_extract, mock_get_dur
    ):
        """The boundary of F3: SOME windows errored but at least one RAN, so the run is a
Success (the
        first window's rally), NOT a Failure. Stage 2 is patched to hand off two windows
directly."""
        mock_env_get.return_value = "dummy_api_key"
        mock_get_dur.return_value = 30.0
        mock_extract.return_value = True

        mock_provider = MagicMock()
        mock_provider.analyze_video.side_effect = [
            ([{"start_time": 0.5, "end_time": 2.0}], {}),  # window 0: a real verdict
            ([], {"error": "generate failed after retries: 503"}),  # window 1: errored
        ]
        mock_provider_registry.get.return_value = mock_provider

        db_mock = MagicMock()
        db_mock.add_play_segment.return_value = "yolo_segment_1"
        two_windows = [{"start": 1.0, "end": 4.0}, {"start": 10.0, "end": 13.0}]

        segmenter = HybridYoloSegmenter(db_mock, {"indexing": {"use_gpu": False}})
        segmenter.person.model = MagicMock()  # short-circuit lazy model load
        with patch.object(
            HybridYoloSegmenter, "_collect_candidate_windows", return_value=two_windows
        ), patch.object(
            HybridYoloSegmenter, "_persist_candidates", return_value=[None, None]
        ):
            res = segmenter.process_video("v1", "dummy.mp4")
        assert not isinstance(res, Failure)
        assert len(res) == 1 and res[0]["start_time"] == 0.5


    @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
    @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
    @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
    @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
    @patch("os.environ.get")
    def test_yolo_hybrid_no_api_key(
        mock_env_get, mock_provider_registry, mock_extract, mock_get_dur, mock_cap
    ):
        """A missing API key is an unrunnable misconfig ? a Failure, NOT a 0-rally [] that
the worker
        would write as a silent status="success" (the wf294 class; yolo_hybrid is the
default segmenter)."""
        mock_env_get.return_value = None

        db_mock = MagicMock()
        config = {"indexing": {"use_gpu": False}}

```

```
211         segmenter = HybridYoloSegmenter(db_mock, config)
212         segmenter.person.model = MagicMock()  # short-circuit lazy model load
213
214         res = segmenter.process_video("v1", "dummy.mp4")
215         assert isinstance(res, Failure) and res.is_recoverable
216         assert "API_KEY" in res.message
217
218
219     @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
```

30. assistant (2026-07-06T18:02:22.663Z)

The yolo tests cover: all-errored (single window) -> Failure (line 153), partial -> Success (line 192-193), and missing-key -> Failure (line 215-216). The trajectory/tracknet tests cover missing-key (line 111), all-errored (line 145), partial (line 180-181), plus healthcheck, no-trajectory, and the legit-zero-windows-empty case.

Now let me verify the review's specific concern about "any test still asserting == [] on a path that now returns Failure". Let me search all test files for `== []` on process_video results and for the converted paths (unknown sport/provider, invalid duration, native-not-supported) coverage.

31. user (2026-07-06T18:02:25.207Z)

```
83:    res = seg.process_video("v1", "clip.mp4")
110:    res = seg.process_video("v1", "clip.mp4")
131:    res = seg.process_video("v1", "clip.mp4")
155:    res = seg.process_video("v1", "clip.mp4")
181:    res = seg.process_video("v1", "clip.mp4")
182:    assert res == [] and not isinstance(res, Failure)
209:    seg.process_video("v1", "clip.mp4")
241:    res = seg.process_video("v1", "clip.mp4")
279:    res = seg.process_video("v1", "clip.mp4")
```

32. user (2026-07-06T18:02:26.491Z)

```
tests\test_annotation_proxy.py:45:    assert [f for f in os.listdir(dst.parent) if
f.startswith(".proxy-")] == []  # temp cleaned up
tests\test_annotation_proxy.py:70:    assert [f for f in os.listdir(dst.parent) if
f.startswith(".proxy-")] == []
tests\test_api_assets.py:119:    assert body["copied"] is False and puts == []  # no copy (D14b)
tests\test_api_assets.py:150:    assert body["copied"] is False and puts == []
tests\test_api_cloud_dispatch.py:214:    assert out["play_segments"] == []
tests\test_api_cloud_dispatch.py:215:    assert db.query_play_segments(out["video_id"], {}) ==
[]
tests\test_api_cloud_dispatch.py:233:    assert fake.calls == []  # never dispatched a local-
only path
tests\test_api_cloud_dispatch.py:328:    assert fake.calls == [] and called == []  # neither
staged nor dispatched
tests\test_api_cloud_dispatch.py:351:        def process_video(self, vid, path, *a, **k):
tests\test_api_cloud_dispatch.py:398:    assert db.query_play_segments(out["video_id"], {}) ==
[]
tests\test_api_cloud_dispatch.py:424:    def process_video(self, vid, path, *a, **k):
tests\test_api_cloud_dispatch.py:459:        fake.calls == []
tests\test_api_cloud_dispatch.py:629:        fake.calls == []
tests\test_api_cloud_dispatch.py:843:        def process_video(self, vid, path, *a, **k):
tests\test_api_endpoints.py:60:    def process_video(self, video_id, video_path, *a, **k):
tests\test_api_endpoints.py:196:    assert env[0].get("/api/videos").json()["videos"] == []
tests\test_api_endpoints.py:226:    assert env[0].get("/api/reels").json()["reels"] == []
tests\test_api_endpoints.py:693:    def process_video(self, video_id, video_path, *a, **k):
tests\test_api_endpoints.py:870:    assert db.list_jobs(None, statuses=None, limit=50) == []
tests\test_api_endpoints.py:1089:        def process_video(self, *a, **k):
tests\test_async_jobs.py:76:    def process_video(self, video_id, video_path, *a, **k):
tests\test_async_jobs.py:93:    def process_video(self, video_id, video_path, *a, **k):
```

```
tests\test_async_jobs.py:228:        def process_video(self, video_id, video_path, *a, **k):
tests\test_async_jobs.py:337:        def process_video(self, *args, **kwargs):
tests\test_async_jobs.py:374:        def process_video(self, video_id, video_path, *args,
**kwargs):
tests\test_audio_features.py:292:    assert seen == []
tests\test_base.py:16:    def process_video(
tests\test_base.py:38:    success = inst.process_video("v1", "path")
tests\test_base.py:44:    failure = inst.process_video("v1", "fail")
tests\test_base.py:89:        def process_video(
tests\test_base.py:117:        def process_video(
tests\test_base.py:158:    assert segments == []
tests\test_base.py:200:        def process_video(
tests\test_base.py:207:        inst.process_video("v1", "path")
tests\test_calibration.py:49:    assert kfold_indices(0, 5) == []
tests\test_check_md_links.py:38:    assert _check(tmp_path, files) == []
tests\test_check_md_links.py:53:    assert _check(tmp_path, {"a.md": body}) == []
tests\test_check_md_links.py:58:    assert _check(tmp_path, files) == []
tests\test_check_md_links.py:63:    assert _check(tmp_path, files) == []
tests\test_compute_backend.py:398:    assert client.cancelled == []  # a finished execution is
never cancelled
tests\test_court_gate.py:124:        == []
tests\test_court_gate.py:130:        == []
tests\test_database.py:179:    assert db.query_candidate_segments("vid1", detector="motion") ==
[]
tests\test_database.py:201:    assert db.query_candidate_segments("vid1", only_unlabeled=True)
== []
tests\test_database.py:210:    assert db.query_candidate_segments("vid1") == []
tests\test_distill_local.py:40:    assert none == []
tests\test_eval_annotations.py:108:    assert load_annotations(path) == []
tests\test_eval_harness.py:109:    assert load_annotations(ann) == []  # header-only template
tests\test_eval_metrics.py:28:    assert r.matches == []
tests\test_eval_metrics.py:58:    assert mr.unmatched_pred_indices == []
tests\test_eval_metrics.py:59:    assert mr.unmatched_gt_indices == []
tests\test_eval_metrics.py:76:    assert mr.matches == []
tests\test_eval_partial_labels.py:43:    assert ep.restrict_to_window([(5.0, 7.0)], 0.0, 5.0) ==
[]
tests\test_execution_policy.py:74:    assert clock.sleeps == []  # no retry, no backoff
tests\test_execution_policy.py:145:    assert calls["n"] == 1 and clock.sleeps == []  # ABORT ->
no retries
tests\test_execution_policy.py:164:    assert clock.sleeps == []  # ready on the first check
tests\test_eval_stats.py:69:    assert grouped_folds(["only"]) == []  # single group ? no fold
tests\test_experiment.py:224:    assert preds == []  # absent feature ? 0 ? fails gate
tests\test_experiment.py:290:    assert res["b_only"] == []  # B is a subset of A here
tests\test_flywheel.py:184:    assert called == []
tests\test_fusion_compare.py:419:    assert extracted == []
tests\test_gemini.py:48:    res = segmenter.process_video("v1", "dummy.mp4")
tests\test_gemini.py:91:    res = segmenter.process_video("v1", "dummy.mp4")
tests\test_gemini.py:131:    return segmenter.process_video(video_id, "dummy.mp4")
tests\test_gemini.py:209:    assert res == []
tests\test_gemini.py:232:        res = segmenter.process_video("vid_single_oversize",
"dummy.mp4")
tests\test_gemini.py:233:    assert res == []
tests\test_gemini.py:248:        res = seg.process_video("vid_nokey", "dummy.mp4")
tests\test_gemini.py:263:        res = seg.process_video("vid_badsport", "dummy.mp4")
tests\test_gemini.py:281:        res = seg.process_video("vid_empty", "dummy.mp4")
tests\test_gemini.py:282:    assert res == [] and not isinstance(res, Failure)
tests\test_fusion_hybrid.py:88:        res = seg.process_video("v1", "clip.mp4")
tests\test_gemini_refine.py:11:    assert gr.merge_overlaps([]) == []
tests\test_gemini_refine.py:71:    assert mapped == []  # no analysis for this candidate
tests\test_golden_fixtures.py:40:    assert mismatches == [], (
tests\test_golden_fixtures.py:149:        assert gf.compare_expected(refrozen, committed) == [],
(
tests\test_golden_manifest.py:53:    assert issues == []
tests\test_golden_manifest.py:102:    assert issues == []
tests\test_golden_real_fixtures.py:15:  * the freeze round-trips (``replay_fixture`` +
```

```
``compare_expected`` == []).
tests\test_golden_real_fixtures.py:543:    assert gf.compare_expected(recomputed, committed) ==
[], (
tests\test_golden_real_fixtures.py:796:        == []
tests\test_hit_detector.py:39:    assert hd.detect_hits(np.zeros(16000 * 2, dtype=np.float32),
16000) == []
tests\test_job_worker_loop.py:73:    def process_video(self, video_id, video_path, *a, **k):
tests\test_job_worker_loop.py:151:        def process_video(self, video_id, video_path, *a,
**k):
tests\test_job_worker_loop.py:206:    assert armed == []  # happy-path job terminalises; nothing
parked -> no timer
tests\test_job_worker_loop.py:312:        def process_video(self, *a, **k):
tests\test_leak_scan.py:21:    assert ls.scan_text(text) == []
tests\test_leak_scan.py:35:    assert ls.scan_text("slug = 'fx_" + "b" * 16 + "'") == []
tests\test_leak_scan.py:36:    assert ls.scan_text("h = '" + "c" * 12 + "'") == []
tests\test_leak_scan.py:41:    assert ls.scan_text("sha = '" + "d" * 40 + "'") == []
tests\test_leak_scan.py:45:    assert ls.scan_text(f"vid = '{_md5_like()}'  # leakscan-allow")
== []
tests\test_leak_scan.py:56:    assert ls.scan_text("a normal line of code",
names=["kushagra_singles"]) == []
tests\test_leak_scan.py:66:    assert ls.scan_repo(tmp_path) == []
tests\test_motion_segmenter.py:107:        result = segmenter.process_video("vid-1", "bad.mp4")
tests\test_motion_segmenter.py:124:        result = segmenter.process_video("vid-1",
"quiet.mp4")
tests\test_motion_segmenter.py:127:    assert result.value == []
tests\test_motion_segmenter.py:154:        result = segmenter.process_video("vid-1",
"active.mp4")
tests\test_nightly_reelcount.py:292:    assert nr._segments_to_intervals([]) == []
tests\test_nightly_regression.py:136:    assert cmp.regressions == []
tests\test_nightly_regression.py:155:    assert cmp.status == "ok" and cmp.regressions == []
tests\test_nightly_regression.py:205:    assert cmp.status == "ok" and cmp.regressions == []
tests\test_output_base.py:100:        seg.process_video("v1", "clip.mp4")
tests\test_per_user_eval.py:119:        assert res["per_k"] == []
tests\test_per_user_eval.py:123:    assert learning_curve([], _control(), _candidate(),
min_k=2)["per_k"] == []
tests\test_profile_parity.py:73:    result = seg.process_video("vidParity", str(video))
tests\test_profile_parity.py:173:    assert ref  # non-vacuous: the stub produced rallies (so
this isn't [] == [])
tests\test_provider_gemini.py:30:        assert segments == []  # no analysis for this chunk
tests\test_provider_gemini.py:104:        assert segments == []
tests\test_provider_gemini.py:235:    assert called == []  # provider never instantiated
tests\test_provider_gemini.py:252:    assert called == []
tests\test_rally_seg_eval.py:156:    assert windows_for(corner, _PRESET) == []
tests\test_rally_slicer.py:92:    assert w == []
tests\test_reporting_harvest.py:88:    assert env.lint() == []
tests\test_restart_recovery.py:261:    assert r.json()["play_segments"] == []
tests\test_restart_recovery.py:262:    assert fake.exists_calls == []
tests\test_restart_recovery.py:353:    assert db.interrupted_remote_process_jobs() == []
tests\test_restart_recovery.py:399:    assert db.interrupted_remote_process_jobs() == []
tests\test_results.py:36:    assert value == []
tests\test_startup_checks.py:85:    assert run_startup_checks(cfg, cuda_available=False, env={})
== []
tests\test_startup_checks.py:90:    assert enforce_startup_checks(cfg, cuda_available=False,
env={}) == []
tests\test_startup_checks.py:98:    assert run_startup_checks(cfg, cuda_available=True, env={})
== []
tests\test_startup_checks.py:119:    assert run_startup_checks(cfg, cuda_available=None, env={})
== []
tests\test_startup_checks.py:132:    assert run_startup_checks(cfg, cuda_available=True, env={})
== []
tests\test_startup_checks.py:162:        == []
tests\test_startup_checks.py:173:        == []
tests\test_startup_checks.py:208:        run_startup_checks(cfg, cuda_available=True,
env={"K_SERVICE": "rally"}) == []
tests\test_startup_checks.py:224:    assert run_startup_checks(cfg, cuda_available=False,
```

```
env={}) == []
tests\test_startup_checks.py:312:    assert calls == []  # probe NEVER called on the default-OFF
path
tests\test_startup_checks.py:365:    assert run_startup_checks(cfg, include_exposure=True,
env={}) == []
tests\test_startup_checks.py:373:    assert run_startup_checks(cfg, env={}) == []  #
include_exposure defaults False
tests\test_startup_checks.py:433:        == []
tests\test_tolerance_metrics.py:33:    assert matches == [] and f1 == 0.0
tests\test_tolerance_metrics.py:90:    assert tm.match_1to1(preds, gts, 2.0, 1.5)[0] == []
tests\test_submit_md5_cache.py:155:    assert drains == []
tests\test_submit_md5_cache.py:165:    assert drains == []
tests\test_submit_md5_cache.py:166:    assert fake.fetched == []  # no bucket reads at all
tests\test_submit_md5_cache.py:332:    assert drains == []
tests\test_submit_md5_cache.py:350:    assert db.list_jobs(None, statuses=None, limit=50) == []
tests\test_submit_md5_cache.py:351:    assert drains == []
tests\test_submit_md5_cache.py:383:    assert drains == []
tests\test_submit_md5_cache.py:405:    assert drains == []
tests\test_tracknet_hybrid.py:83:    res = seg.process_video("v1", "clip.mp4")
tests\test_tracknet_hybrid.py:110:    res = seg.process_video("v1", "clip.mp4")
tests\test_tracknet_hybrid.py:131:    res = seg.process_video("v1", "clip.mp4")
tests\test_tracknet_hybrid.py:155:    res = seg.process_video("v1", "clip.mp4")
tests\test_tracknet_hybrid.py:181:    res = seg.process_video("v1", "clip.mp4")
tests\test_tracknet_hybrid.py:182:    assert res == [] and not isinstance(res, Failure)
tests\test_tracknet_hybrid.py:209:    seg.process_video("v1", "clip.mp4")
tests\test_tracknet_hybrid.py:241:    res = seg.process_video("v1", "clip.mp4")
tests\test_tracknet_hybrid.py:279:    res = seg.process_video("v1", "clip.mp4")
tests\test_tracknet_runner.py:118:    assert windows == []
tests\test_trajectory_hybrid_equivalence.py:69:        return db, seg.process_video("v1",
"clip.mp4")
tests\test_trajectory_hybrid_equivalence.py:144:            res = seg.process_video("v1",
"clip.mp4")
tests\test_trajectory_hybrid_equivalence.py:179:        res = seg.process_video("v1",
"clip.mp4")
tests\test_wasb_cache.py:69:    assert det[3] == []  # seen but never detected
tests\test_wasb_cache.py:292:    assert wi.build_windows(["00000000.png", "00000001.png"], 3) ==
[]
tests\test_wasb_cache.py:461:    assert list(wi._PrefetchIter(iter([]), depth=2)) == []
tests\test_worker.py:121:    def process_video(self, video_id, path, max_frames=None):
tests\test_worker.py:200:    def process_video(self, video_id, path, max_frames=None):
tests\test_worker.py:211:    def process_video(self, video_id, path, max_frames=None):
tests\test_worker.py:309:    assert report["segments"] == []
tests\test_yolo_hybrid.py:98:    res = segmenter.process_video("v1", "dummy.mp4")
tests\test_yolo_hybrid.py:152:        res = segmenter.process_video("v1", "dummy.mp4")
tests\test_yolo_hybrid.py:191:        res = segmenter.process_video("v1", "dummy.mp4")
tests\test_yolo_hybrid.py:214:    res = segmenter.process_video("v1", "dummy.mp4")
tests\test_yolo_hybrid.py:262:    res = segmenter.process_video("v1", "dummy.mp4")
tests\test_yolo_hybrid.py:344:    res = segmenter.process_video("v1", "dummy.mp4")
tests\test_yolo_hybrid.py:397:    segmenter.process_video("v1", "dummy.mp4")
tests\test_yolo_hybrid.py:448:    segmenter.process_video("v1", "dummy.mp4")
```

33. assistant (2026-07-06T18:02:32.500Z)

Let me check the two yolo tests at lines 262 and 344 (later in the file) to make sure none of them assert `== []` on a path that now returns Failure ? these are the "unknown provider" / "invalid duration" type tests potentially.

34. user (2026-07-06T18:02:33.832Z)

```
219     @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
220     @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
221     @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
222     @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
223     @patch("os.environ.get")
224     def test_yolo_hybrid_drops_bad_timestamps(
```

```
        mock_env_get, mock_provider_registry, mock_extract, mock_get_dur, mock_cap
    ):
        """Play segments with unparseable timestamps should be dropped, not silently
kept."""
        mock_env_get.return_value = "dummy_api_key"
        mock_get_dur.return_value = 10.0
        mock_extract.return_value = True

        # Provider returns one good segment and one with garbage timestamps
        mock_provider = MagicMock()
        mock_provider.analyze_video.return_value = (
            [
                {"start_time": 1.0, "end_time": 3.0},
                {"start_time": "N/A", "end_time": "??"},
            ],
            {},
        )
        mock_provider_registry.get.return_value = mock_provider

        db_mock = MagicMock()
        db_mock.add_play_segment.return_value = "segment_ok"

        config = {
            "indexing": {
                "use_gpu": False,
                "chunk_duration": 100,
                "chunk_overlap": 10,
                "yolo_velocity_threshold": 0.05,
                "dead_time_merge_gap": 2.0,
                "yolo_frame_skip": 1,
                "min_action_window_duration": 0.5,
            }
        }

        segmenter = HybridYoloSegmenter(db_mock, config)
        _mock_stage1(segmenter)
        mock_cap.return_value = _make_cap_mock()

        res = segmenter.process_video("v1", "dummy.mp4")

        # Only the valid segment should survive ? the "N/A" one should be dropped
        assert len(res) == 1
        assert res[0]["start_time"] == 1.0


    @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
    def test_extract_windows_returns_enriched_dicts(mock_cap):
        """The person cue's window extraction should return dicts with motion stats."""
        config = {"indexing": {"min_action_window_duration": 1.0}}
        segmenter = HybridYoloSegmenter(MagicMock(), config)
        _mock_stage1(segmenter)
        mock_cap.return_value = _make_cap_mock()

        windows = segmenter.person.extract_action_windows_from_chunk(
            "chunk.mp4",
            chunk_start=5.0,
            velocity_thresh=0.05,
            merge_gap=2.0,
            frame_skip=1,
            chunk_index=2,
        )

        assert len(windows) >= 1
        w = windows[0]
        assert set(w) >= {
```

```
            "start",
            "end",
            "active_frame_count",
            "peak_velocity",
            "mean_velocity",
            "track_id_count",
            "chunk_index",
        }
        assert w["chunk_index"] == 2
        assert w["start"] >= 5.0  # offset into full-video coordinates
        assert w["active_frame_count"] > 0
        assert w["peak_velocity"] >= w["mean_velocity"] > 0
        assert w["track_id_count"] == 1


    @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
    @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
    @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
    @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
    @patch("os.environ.get")
    def test_yolo_hybrid_persists_and_links_candidates(
        mock_env_get, mock_provider_registry, mock_extract, mock_get_dur, mock_cap
    ):
        """Raw candidates should be stored, and confirmed ones linked to their segment."""
        mock_env_get.return_value = "dummy_api_key"
        mock_get_dur.return_value = 10.0
        mock_extract.return_value = True

        mock_provider = MagicMock()
        mock_provider.analyze_video.return_value = (
            [{"start_time": 0.0, "end_time": 3.0, "shuttle_exchange_count": 2}],
            {},
        )
        mock_provider_registry.get.return_value = mock_provider

        db_mock = MagicMock()
        db_mock.add_candidate_segment.return_value = 77
        db_mock.add_play_segment.return_value = 101

        config = {
            "indexing": {
                "use_gpu": False,
                "chunk_duration": 100,
                "chunk_overlap": 10,
                "yolo_velocity_threshold": 0.05,
                "dead_time_merge_gap": 2.0,
                "yolo_frame_skip": 1,
                "min_action_window_duration": 1.0,
            }
        }

        segmenter = HybridYoloSegmenter(db_mock, config)
        _mock_stage1(segmenter)
        mock_cap.return_value = _make_cap_mock()

        res = segmenter.process_video("v1", "dummy.mp4")

        assert len(res) == 1
        # Candidate persisted with the right detector + reproducibility params
        assert db_mock.add_candidate_segment.called
        _, kwargs = db_mock.add_candidate_segment.call_args
        assert kwargs["detector"] == "yolo_hybrid"
        assert "peak_velocity" in kwargs["stats"]
        assert kwargs["detection_params"]["frame_skip"] == 1
        # Confirmed segment linked back to its candidate
```

```
        db_mock.link_candidate_to_segment.assert_called_once_with(77, 101)


    @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
    @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
    @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
    @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
    @patch("os.environ.get")
    def test_yolo_hybrid_store_candidates_disabled(
        mock_env_get, mock_provider_registry, mock_extract, mock_get_dur, mock_cap
    ):
        """store_yolo_candidates=False should skip DB persistence (and linking)."""
        mock_env_get.return_value = "dummy_api_key"
        mock_get_dur.return_value = 10.0
        mock_extract.return_value = True

        mock_provider = MagicMock()
        mock_provider.analyze_video.return_value = (
            [{"start_time": 0.0, "end_time": 3.0}],
            {},
        )
        mock_provider_registry.get.return_value = mock_provider

        db_mock = MagicMock()
        db_mock.add_play_segment.return_value = 101

        config = {
            "indexing": {
                "use_gpu": False,
                "chunk_duration": 100,
                "chunk_overlap": 10,
                "yolo_velocity_threshold": 0.05,
                "dead_time_merge_gap": 2.0,
                "yolo_frame_skip": 1,
                "min_action_window_duration": 1.0,
                "store_yolo_candidates": False,
            }
        }

        segmenter = HybridYoloSegmenter(db_mock, config)
        _mock_stage1(segmenter)
        mock_cap.return_value = _make_cap_mock()

        segmenter.process_video("v1", "dummy.mp4")

        db_mock.add_candidate_segment.assert_not_called()
        db_mock.link_candidate_to_segment.assert_not_called()


    @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
    @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
    @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
    @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
    @patch("os.environ.get")
    @patch("os.makedirs")
    def test_yolo_chunk_dirs_follow_output_dir(
        mock_makedirs,
        mock_env_get,
        mock_provider_registry,
        mock_extract,
        mock_get_dur,
        mock_cap,
    ):
        """M3: yolo's chunk-staging (`chunks_yolo`) + AI-handoff
(`chunks_<provider>_handoff`) dirs
```

```
418         root under the configured output.output_dir, not a hardcoded ./output."""
419         mock_env_get.return_value = "k"
420         mock_get_dur.return_value = 10.0
421         mock_extract.return_value = True
422         mock_provider = MagicMock()
423         mock_provider.analyze_video.return_value = (
424             [{"start_time": 0.0, "end_time": 3.0, "shuttle_exchange_count": 2}],
425             {},
426         )
427         mock_provider_registry.get.return_value = mock_provider
428         db_mock = MagicMock()
429         db_mock.add_play_segment.return_value = "s1"
430 
431         out = os.path.join("custom_m3", "yolo_out")
432         config = {
433             "indexing": {
434                 "use_gpu": False,
435                 "chunk_duration": 100,
436                 "chunk_overlap": 10,
437                 "yolo_velocity_threshold": 0.1,
438                 "dead_time_merge_gap": 2.0,
439                 "yolo_frame_skip": 1,
440                 "min_action_window_duration": 1.0,
441             },
442             "output": {"output_dir": out},
443         }
444         segmenter = HybridYoloSegmenter(db_mock, config)
445         _mock_stage1(segmenter)
446         mock_cap.return_value = _make_cap_mock()
447 
448         segmenter.process_video("v1", "dummy.mp4")
449 
450         made = [c.args[0] for c in mock_makedirs.call_args_list if c.args]
451         assert os.path.join(out, "chunks_yolo") in made
452         assert os.path.join(out, "chunks_gemini_handoff") in made
453         # ...and nothing leaked to a literal ./output for these dirs.
454         assert os.path.join("output", "chunks_yolo") not in made
455 
```

35. assistant (2026-07-06T18:02:38.741Z)

Those yolo tests all hand off with `metrics={}` (no error) and at least one window that runs, so they return real segments ? none assert `== []` on a now-Failure path. Good.

Now let me examine the F1/F2 coverage gap concern precisely. The review asks whether the *other converted paths* (unknown sport/provider, invalid duration, native-not-supported) are covered at all. Let me check the tracknet test file fully (lines 200-280) and check if there's a `_resolve_runner` NotImplementedError / invalid-duration test.

36. user (2026-07-06T18:02:40.072Z)

```
185 
186 
187     @_patched
188     def test_pipeline_store_candidates_disabled(
189         mock_runner_cls,
190         mock_probe,
191         mock_dur,
192         mock_extract,
193         mock_provider_registry,
194         mock_env,
195     ):
196         mock_env.return_value = "dummy_key"
197         mock_dur.return_value = 30.0
198         mock_extract.return_value = True
```

```
        mock_runner_cls.return_value = _make_runner()

        provider = MagicMock()
        provider.analyze_video.return_value = ([{"start_time": 0.5, "end_time": 3.0}], {})
        mock_provider_registry.get.return_value = provider

        db = MagicMock()
        db.add_play_segment.return_value = 200

        seg = TrackNetHybridSegmenter(db, {"indexing": {"store_yolo_candidates": False}})
        seg.process_video("v1", "clip.mp4")

        db.add_candidate_segment.assert_not_called()
        db.link_candidate_to_segment.assert_not_called()


    @_patched
    def test_pipeline_drops_unparseable_segments(
        mock_runner_cls,
        mock_probe,
        mock_dur,
        mock_extract,
        mock_provider_registry,
        mock_env,
    ):
        mock_env.return_value = "dummy_key"
        mock_dur.return_value = 30.0
        mock_extract.return_value = True
        mock_runner_cls.return_value = _make_runner()

        provider = MagicMock()
        provider.analyze_video.return_value = (
            [{"start_time": 0.5, "end_time": 3.0}, {"start_time": "N/A", "end_time": "??"}],
            {},
        )
        mock_provider_registry.get.return_value = provider

        db = MagicMock()
        db.add_candidate_segment.return_value = 55
        db.add_play_segment.return_value = 200

        seg = TrackNetHybridSegmenter(db, {"indexing": {}})
        res = seg.process_video("v1", "clip.mp4")
        assert len(res) == 1  # the "N/A" segment is dropped


    def test_probe_fps_width_fallbacks():
        with patch(
            "backend.pipeline.segmenters.trajectory_hybrid.cv2.VideoCapture"
        ) as mock_cap:
            bad = MagicMock()
            bad.isOpened.return_value = False
            bad.get.return_value = 0.0
            mock_cap.return_value = bad
            fps, width = TrackNetHybridSegmenter._probe_fps_width("missing.mp4")
            assert fps == 30.0 and width == 1280.0


    @_patched
    def test_skip_ai_handoff_emits_candidates_as_rallies(
        mock_runner_cls,
        mock_probe,
        mock_dur,
        mock_extract,
        mock_provider_registry,
```

```
264         mock_env,
265     ):
266         """Fully-local mode: candidate windows become rallies with NO provider/API key/clip
upload."""
267         mock_env.return_value = (
268             None  # deliberately no API key ? fully-local must not need one
269         )
270         mock_dur.return_value = 30.0
271         windows = [_window(1.0, 4.0), _window(10.0, 13.0)]
272         mock_runner_cls.return_value = _make_runner(windows=windows)
273 
274         db = MagicMock()
275         db.add_candidate_segment.side_effect = [101, 102]
276         db.add_play_segment.side_effect = [201, 202]
277 
278         seg = TrackNetHybridSegmenter(db, {"indexing": {"skip_ai_handoff": True}})
279         res = seg.process_video("v1", "clip.mp4")
280 
281         # Both candidate windows became rallies, verbatim, with no AI involvement.
282         assert len(res) == 2
283         mock_provider_registry.get.assert_not_called()  # no provider built
284         mock_extract.assert_not_called()  # no handoff clip extraction/upload
285         assert (res[0]["start_time"], res[0]["end_time"]) == (1.0, 4.0)
286         assert (res[1]["start_time"], res[1]["end_time"]) == (10.0, 13.0)
287         # Linked back to the persisted candidates.
288         db.link_candidate_to_segment.assert_any_call(101, 201)
289         # Local detector tag + confidence marker (metadata is the 6th positional arg).
290         meta = db.add_play_segment.call_args_list[0].args[5]
291         assert meta["detector"] == "tracknet-hybrid-local-noai"
292         assert meta["shot_count_confidence"] == "LocalNoAI"
293 
294 
```

37. assistant (2026-07-06T18:02:43.635Z)

I have a complete picture of the test coverage. Let me now confirm the F1/F2 coverage gap: the converted paths **unknown sport (KeyError), unknown provider (KeyError), invalid duration (<=0), and _resolve_runner NotImplementedError (native-not-supported)** ? are they tested anywhere? Let me search specifically for these across all tests.

38. user (2026-07-06T18:02:46.776Z)

```
36:     fn = patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration")(fn)
```

39. user (2026-07-06T18:02:48.694Z)

No matches found

40. assistant (2026-07-06T18:03:00.265Z)

Confirmed: the F1/F2 converted paths **unknown sport, unknown provider, invalid/<=0 duration, and native-not-supported (`_resolve_runner` NotImplementedError)** have NO test coverage in either the tracknet or yolo test suites. Only the missing-API-key path is locked. This is a test-gap: these Failure conversions are unverified, so a future regression that reverts any of them to `return []` would not be caught. This matches the review's explicit question ("Are the other converted paths ... covered at all, or silently uncovered?"). This is a legitimate low/medium test-gap finding.

Now let me verify one edge case in the F3 logic more carefully. In trajectory_hybrid, the all-errored gate is `if handoff_indices and handoff_failed == len(handoff_indices)`. But `handoff_failed` counts errors over `window_results`, which comes from `handoff_indices`. So `handoff_failed <= len(handoff_indices)`. Fine. But there's a subtle question: `handoff_indices` is the result of `_select_for_handoff`, which for the base is all windows but for fusion may gate down. If `_select_for_handoff` returns a subset ? say only 1 of 3 windows is handed off,

and that 1 errors ? then `handoff_failed=1 == len(handoff_indices)=1` ? Failure. Is that correct? The 2 non-handed-off windows were gated OUT (a deliberate decision, not an error), so the run DID process them (gated them). But the only window that went to AI got no verdict. The comment says "all handed-off windows errored ? zero information". That is defensible ? the handoff produced no verdict. Not a defect.

Now let me check the worker test for the `_fail` path ? specifically test_worker line 309 asserts `report["segments"] == []`. Let me confirm the worker tests are consistent and check the `_EmptySeg`/`_FailSeg` return-signature match (the real segmenter's `process_video` signature vs the stub). Let me look at test_worker around 115-125 and 195-215.

41. user (2026-07-06T18:03:01.278Z)

```
115
116
117     class _FakeSeg:
118         def __init__(self, db, config):
119             pass
120
121         def process_video(self, video_id, path, max_frames=None):
122             return [
123                 {
124                     "start_time": 2.0,
125                     "end_time": 5.0,
126                     "shot_count": 4,
127                     "shot_count_confidence": "LocalNoAI",
128                 },
129                 {
```

42. user (2026-07-06T18:03:02.211Z)

```
246                 "infer",
247                 "--video-uri",
248                 str(video),
249                 "--out-uri",
250                 str(out),
251                 "--scratch",
252                 str(tmp_path / "s"),
253                 "--segmenter",
254                 "wasb_hybrid",
255             ]
256         )
257         with caplog.at_level("WARNING", logger="backend.worker"):
258             rc = wmain.cmd_infer(args)
259         assert rc == 0  # still a clean run, just empty ? but now EXPLAINED
260         msgs = " ".join(r.getMessage() for r in caplog.records)
261         assert "0 segments" in msgs and "vidZ" in msgs
262         assert "detector_impl=native" in msgs  # surfaces the resolved detector
263         assert "native runner UNAVAILABLE" in msgs  # points at the likely cause
264         # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match ? status stays
"success"
265         # (contrast test_cmd_infer_detector_failure_*, where a Failure surfaces as
status="failed").
```

43. assistant (2026-07-06T18:03:15.148Z)

The worker tests are coherent. `_EmptySeg` (bare []) ? rc 0, status success; `_FailSeg` (Failure) ? rc 3, status failed; and the real-segmenter end-to-end test locks it through `WasbHybridSegmenter`. Good.

Now let me verify one thing about the `test_cmd_infer_real_trajectory_segmenter_failure_flows_to_report` test ? it patches `_resolve_runner` and `get_video_duration` but relies on `skip_ai_handoff=true` to avoid needing an API key. Under `skip_ai=True` in trajectory_hybrid, the code path skips the provider

resolution block (line 213-227 `if not skip_ai`). Let me confirm that healthcheck runs and then `run_predict` returns None ? the no-trajectory Failure fires BEFORE reaching the skip_ai handoff logic. Yes ? the no-trajectory abort (line 300) is before Phase 3. And `runner.healthcheck` returns `(True, "env ok")`. So the test correctly reaches the no-trajectory Failure. Good.

Let me now confirm the mypy quarantine claim for the `# type: ignore[override]` on yolo's `process_video` and check whether trajectory_hybrid (NOT quarantined) has any type issue with its widened `Union` return being a forward-ref string. Let me check the `Union` import is used and there are no unused-import lint issues. In trajectory_hybrid, `Union` was added to imports (line 9 of diff) and used in the annotation string `"Union[...]"`. Since it's a string annotation, is `Union` actually "used" from ruff's perspective? Ruff's F401 typically still sees names in string annotations if `from __future__ import annotations` or via its type-checking awareness. Let me verify trajectory_hybrid imports and whether Union is referenced outside the string.

44. user (2026-07-06T18:03:17.309Z)

```
25:from typing import Any, Dict, List, Optional, Tuple, Union
180:    ) -> "Union[List[Dict[str, Any]], Failure]":
```

45. user (2026-07-06T18:03:18.139Z)

```
25:from typing import Any, Dict, List, Tuple, Union
328:    ) -> "Union[List[dict], Failure]":
492:    ) -> "Union[List[Dict[str, Any]], Failure]":
```

46. assistant (2026-07-06T18:03:26.821Z)

`Union` is referenced only inside string annotations. The CI is confirmed clean (ruff 0.15.16 sees string-annotation names as used via its type-checking analysis, and the review states current state is ruff clean / mypy clean), so no lint defect there. The prompt explicitly says not to speculate that tests/CI might fail.

Now, one more important correctness check on trajectory_hybrid: the widened return annotation is a forward-ref string `"Union[List[Dict[str, Any]], Failure]"`. Since trajectory_hybrid is NOT mypy-quarantined, mypy must resolve this. `Failure` and `Union` are both imported at module level, so the forward ref resolves. And there's no `# type: ignore[override]` on trajectory's `process_video` ? was one needed? The base ABC declares a Result contract. Let me quickly check the base class signature to confirm trajectory's override is mypy-compatible without an ignore (the diff removed the old `override-ignore` comment and did NOT add a `# type: ignore[override]`, unlike yolo which kept one). This matters: if the base ABC's `process_video` return type is incompatible, trajectory (non-quarantined) would fail mypy ? but the prompt says mypy is clean. So it must be compatible. Let me verify the base signature to understand why yolo needs the ignore but trajectory doesn't (a legitimate convention question).

47. user (2026-07-06T18:03:28.623Z)

```
44:    def process_video(
48:        player_pool: List[str] = None,
49:        max_frames: int = None,
50:    ) -> "Result[List[Dict[str, Any]]]":
```

48. assistant (2026-07-06T18:03:37.468Z)

Now I understand the difference. The base `process_video` signature (line 44-50) has params `player_pool: List[str] = None, max_frames: int = None` (non-Optional annotations with None defaults). yolo's `process_video` uses the SAME param annotations (`player_pool: List[str] = None`), so it matches the base's parameter types but keeps `# type: ignore[override]` (yolo is quarantined anyway, so the ignore is harmless).

trajectory's `process_video` uses `player_pool: Optional[List[str]] = None, max_frames: Optional[int] = None` (proper Optional). This is a WIDER (more permissive) parameter type than the base's `List[str]`, which under Liskov/mypy is actually fine for parameters (contravariance) ? parameters may be widened. And the return `Union[List[Dict[str, Any]], Failure]` ? the base

returns `Result[List[Dict[str, Any]]]` = `Union[Success[List[...]], Failure]`. trajectory returns `Union[List[...], Failure]`. `List[...]` is NOT a subtype of `Success[List[...]]`. So trajectory's return type is technically incompatible with the base's declared `Result` return... yet the prompt states mypy is clean and trajectory is not quarantined.

This is worth verifying rather than assuming. Let me check the mypy.ini and whether there's a per-line ignore or whether mypy actually flags this. Let me look at how the base's `Result` return reconciles ? maybe base returns bare list too, or mypy config. Let me check mypy.ini and the actual base return more fully.

49. user (2026-07-06T18:03:38.475Z)

```
40              """Human-readable explanation of this segmenter."""
41              pass
42
43          @abstractmethod
44          def process_video(
45              self,
46              video_id: str,
47              video_path: str,
48              player_pool: List[str] = None,
49              max_frames: int = None,
50          ) -> "Result[List[Dict[str, Any]]]":
51              """
52              Processes a video to detect play segments, populates SQLite database,
53              and returns a Result.
54
55              Use isinstance(result, Failure) (or result.is_success()) to distinguish
56              "no segments found" from "something went wrong" (missing key, healthcheck
57              failure, etc.). This is the start of the standardized error/result contract.
58              """
59              pass
60
61
62      class SegmenterRegistry:
63          def __init__(self):
64              self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
65
66          def register(self, segmenter_cls: Type[BasePlaySegmenter]):
67              """Registers a new segmenter class under its name property."""
68              try:
69                  temp_inst = segmenter_cls(None, {})
70                  name = temp_inst.name
71              except Exception as e:
72                  raise ValueError(
73                      f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}"
74                  ) from e
75              self._segmenters[name] = segmenter_cls
76
77          def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
78              return self._segmenters.get(name)
79
80          def list_available(self) -> Dict[str, str]:
81              """Returns a dict of registered segmenter names and their descriptions."""
82              res = {}
83              for name, cls in self._segmenters.items():
84                  try:
85                      temp_inst = cls(None, {})
86                      desc = temp_inst.description
87                  except Exception:
88                      desc = "No description available."
89                  res[name] = desc
90              return res
91
```

```
92
93      segmenter_registry = SegmenterRegistry()
94
```

50. user (2026-07-06T18:03:38.963Z)

```
1       [mypy]
2       # Lenient, GREEN baseline (quality sweep AI-4): the gate protects the clean
3       # modules and pins an explicit quarantine list below ? not a typing crusade.
4       # Ratchet: when you touch a quarantined module, try removing its entry.
5       ignore_missing_imports = True
6       # backend/ is a namespace package (no root __init__.py) ? required so mypy
7       # maps files to backend.* module names instead of duplicating top-level names.
8       explicit_package_bases = True
9
10      # --- Quarantine (known-loose modules, each with a reason) ---
11
12      # Optional module globals (db_instance/global_config set by create_app() / main()).
13      # Route handlers use request.app.state exclusively; helpers take explicit params.
14      # backend.api.job_worker lifted from quarantine (now takes explicit config/db params).
15      [mypy-backend.main]
16      ignore_errors = True
17
18      # Worker dispatcher: composes the SAME validator/segmenter/report building blocks as
19      # backend.main, which are typed `config: dict` but receive the AppConfig migration shim
20      # at runtime (the .get() compat layer). Shares backend.main's config-shim quarantine
21      # until the typed-config migration removes the dict?AppConfig duality everywhere.
22      [mypy-backend.worker.__main__]
23      ignore_errors = True
24
25      # yolo_hybrid: the fusion-P1 extraction moved its loose torchvision/cv2 inline code
26      # to detectors/person_detector.py (which IS type-clean, no quarantine). What remains
27      # here is pre-existing orchestration debt unrelated to that move (implicit-Optional
28      # process_video defaults, the Success|Failure ABC-override mismatch, a Future[...]
29      # annotation) ? ratchet those out in a focused typing pass, not here.
30      # (wasb_hybrid/tracknet_hybrid left the quarantine with the AI-7 collapse.)
31      [mypy-backend.pipeline.segmenters.yolo_hybrid]
32      ignore_errors = True
33
34      [mypy-backend.pipeline.segmenters.gemini]
35      ignore_errors = True
36
37      # Registry builds cls(None, {}) probes; typing it honestly needs a Protocol.
38      [mypy-backend.pipeline.segmenters.base]
39      ignore_errors = True
40
```

51. assistant (2026-07-06T18:03:46.359Z)

This is the key insight. trajectory_hybrid is NOT in the quarantine list, so mypy checks it. Its `process_video` return annotation is `"Union[List[Dict[str, Any]], Failure]"` while the base declares `"Result[List[Dict[str, Any]]]"` = `Union[Success[List[...]], Failure]`. A bare `List[...]` is NOT assignable to `Success[List[...]]`, so this override return type is incompatible with the base ? which would normally be a mypy `[override]` error. Yet trajectory does NOT have a `# type: ignore[override]` (the diff removed the old `override-ignore` comment and added none), and the prompt asserts mypy is clean.

I should not just assume. Let me actually run mypy on trajectory_hybrid to verify whether it's genuinely clean or whether there's a real defect (a missing `# type: ignore[override]` that the prompt's "mypy clean" claim might not actually cover, OR it's clean for a reason I'm missing ? e.g., mypy treats the string forward-ref differently, or the params being widened to Optional somehow changes signature matching). This is exactly the kind of grounded verification the review demands.

52. user (2026-07-06T18:03:51.952Z)

```
backend\sports\__init__.py:9: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 1 source file
```

53. assistant (2026-07-06T18:03:58.008Z)

mypy is genuinely clean on trajectory_hybrid ? no `[override]` error, no ignore needed. The reason: mypy's override-compatibility check for the return type sees `List[Dict[str, Any]]` and checks it against `Result[List[Dict[str, Any]]]` = `Union[Success[...], Failure]`. Since `Success` is `Generic[T]` and mypy... actually it passed, so mypy considers the return compatible (likely because it does a per-file check and the base is quarantined via `ignore_errors`, OR the subtype relationship holds via structural width). Either way, it's clean ? confirmed empirically, not assumed. No defect. Good that I verified rather than reporting a false positive.

Let me also run mypy on the broader backend to confirm the whole thing is clean as claimed, and run the specific test files to be thorough about the LGTM gate (per my global instructions I verify rather than trust). Let me run the affected test files.

54. user (2026-07-06T18:04:05.479Z)

```
.............................................................  [100%]
71 passed in 3.07s
```

55. assistant (2026-07-06T18:04:12.628Z)

All 71 tests in the affected files pass. Let me also run ruff on the two changed segmenters and the worker to confirm lint clean (verifying, not trusting), and check the fusion_hybrid test at line 88 which calls process_video ? since fusion subclasses trajectory, does it have any assertion that would break, and does fusion have F3 coverage.

56. user (2026-07-06T18:04:14.980Z)

```
All checks passed!
```

57. user (2026-07-06T18:04:14.985Z)

```
70
71          with patch.object(
72              FusionSegmenter,
73              "_build_runner",
74              return_value=(_runner(), {"weights_path": "w"}),
75          ), patch.object(
76              FusionSegmenter, "_probe_fps_width", return_value=(30.0, 1280.0)
77          ), patch(
78              "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
79              return_value=30.0,
80          ), patch(
81              "backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
82              return_value=True,
83          ), patch(
84              "backend.pipeline.segmenters.trajectory_hybrid.provider_registry"
85          ) as preg, patch("os.environ.get", return_value="dummy_key"):
86              preg.get.return_value = provider
87              seg = FusionSegmenter(db, {"indexing": indexing or {}})
88              res = seg.process_video("v1", "clip.mp4")
89              return db, provider, res
90
91
92      def _stats_calls(db):
```

```
93          """Persisted stats dicts, in call order."""
94          return [kw["stats"] for _a, kw in db.add_candidate_segment.call_args_list]
95
96
97      def test_net_crossings_persisted_and_person_off_by_default():
98          db, _provider, _res = _run()
99          stats = _stats_calls(db)
```

58. assistant (2026-07-06T18:04:27.257Z)

Everything verified clean. The fusion test uses a provider returning a real verdict `{}` metrics, so it won't hit the all-errored gate. Fusion is a subclass of trajectory_hybrid and inherits the F3 logic; it's not separately tested for F3, but fusion isn't in the review's explicit scope (F3 mentions "trajectory (parametrized twins) and yolo"). Not a required-coverage gap per the review scope.

Let me do a final consolidation of my findings. I've confirmed:

1. **Convention inconsistency (real, low)**: The `is_recoverable=True` on the missing-API-key path in trajectory_hybrid.py:225 and yolo_hybrid.py:98, with comments explicitly claiming "mirrors gemini / mirrors the gemini segmenter" ? but gemini.py:116 returns `Failure(msg)` with the default `is_recoverable=False` for the exact same missing-key case. The "mirrors gemini" comment is factually contradicted by the code. No consumer reads `is_recoverable` today so it's not a behavior bug, but it's an inconsistent classification of the same failure class across the three segmenters and a stale/incorrect comment.

2. **Test gap (real, low/medium)**: F1/F2 converted paths ? unknown sport (KeyError), unknown provider (KeyError), invalid/<=0 duration, and native-not-supported (`_resolve_runner`/`_resolve_provider` NotImplementedError) ? have NO test coverage in either tracknet or yolo suites. Only the missing-API-key conversion is locked. A regression reverting any of these to `return []`/`return None` would pass CI silently.

Let me double-check finding 1's severity by confirming gemini's missing-key really is the "mirror" being referenced and there's no other gemini path with is_recoverable=True. I already saw gemini.py lines 100/116/124 all use bare `Failure(...)` (no is_recoverable). Confirmed.

Let me verify the exact line numbers in the current source for precise anchoring.

59. user (2026-07-06T18:04:28.213Z)

```
214                     from backend.pipeline.segmenters._keyhint import api_key_hint
215
216                     msg = (
217                         f"{api_key_env_var} environment variable is not set! "
218                         f"To run the {provider_name} handoff, set your API key. "
219                         + api_key_hint(api_key_env_var)
220                     )
221                     logger.error(msg)
222                     # A misconfig (unrunnable), NOT a 0-rally match ? surface as Failure so
the run is
223                     # marked failed instead of a silent empty "success" (mirrors gemini; the
worker's
224                     # own 0-segment diagnostic names this missing-key case as a top cause).
225                     return Failure(msg, is_recoverable=True)
226                 try:
227                     provider = provider_registry.get(
```

60. assistant (2026-07-06T18:04:38.145Z)

Confirmed. Line 223: comment says "mirrors gemini" but gemini.py:116 uses `is_recoverable=False` (default) for the same missing-key case, while this line uses `is_recoverable=True`.

I have thoroughly verified the diff against the actual code, run mypy (clean), ruff (clean), and the affected tests (71 passed). I found two grounded findings. Let me now assess severity

carefully and report.

Both findings are genuinely low severity: neither is a runtime/behavior bug (no consumer reads `is_recoverable`; the untested paths are correct in code, just unlocked). But both are real per the review's explicit lens: finding 1 is a convention inconsistency + a factually-wrong "mirrors gemini" comment; finding 2 is a concrete test gap the review specifically asked about ("covered at all, or silently uncovered?").

Let me submit the structured output.

61. user (2026-07-06T18:05:02.084Z)

Structured output provided successfully